FLUTTERFLIRT · TECHNICAL BLUEPRINT

StockPilot WMS

Stock, Vendor & Warehouse Management System

Architecture · Process Flows · PostgreSQL Backend · Next.js Full-Stack Web Application · Complete Source Code

Next.js 14 | PostgreSQL 16 | TypeScript | REST API | JWT + RBAC

Prepared by Naveen, Founder, FlutterFlirt
www.flutterflirt.com · info@flutterflirt.com
Version 1.0 · 29 October 2026

Contents

1. Overview & Modules
2. Technology Stack
3. System Architecture
4. Roles & Permissions
5. Business Process Flows
6. Inventory Engine (Ledger, Costing, Reservation)
7. Database Design
8. REST API Reference
9. Project Structure
10. Application Screens
11. Setup & Deployment
12. Appendix A — PostgreSQL Schema & Seed
13. Appendix B — Complete Next.js Source Code

> **Status:** the application in this document has been built, compiled (`next build`) and tested end-to-end against a live PostgreSQL 16 database: login and role checks, PO approve → partial GRN → full GRN, sales order reserve → dispatch, transfers, adjustments, vendor payments and negative-stock protection all pass.

1. Overview & Modules

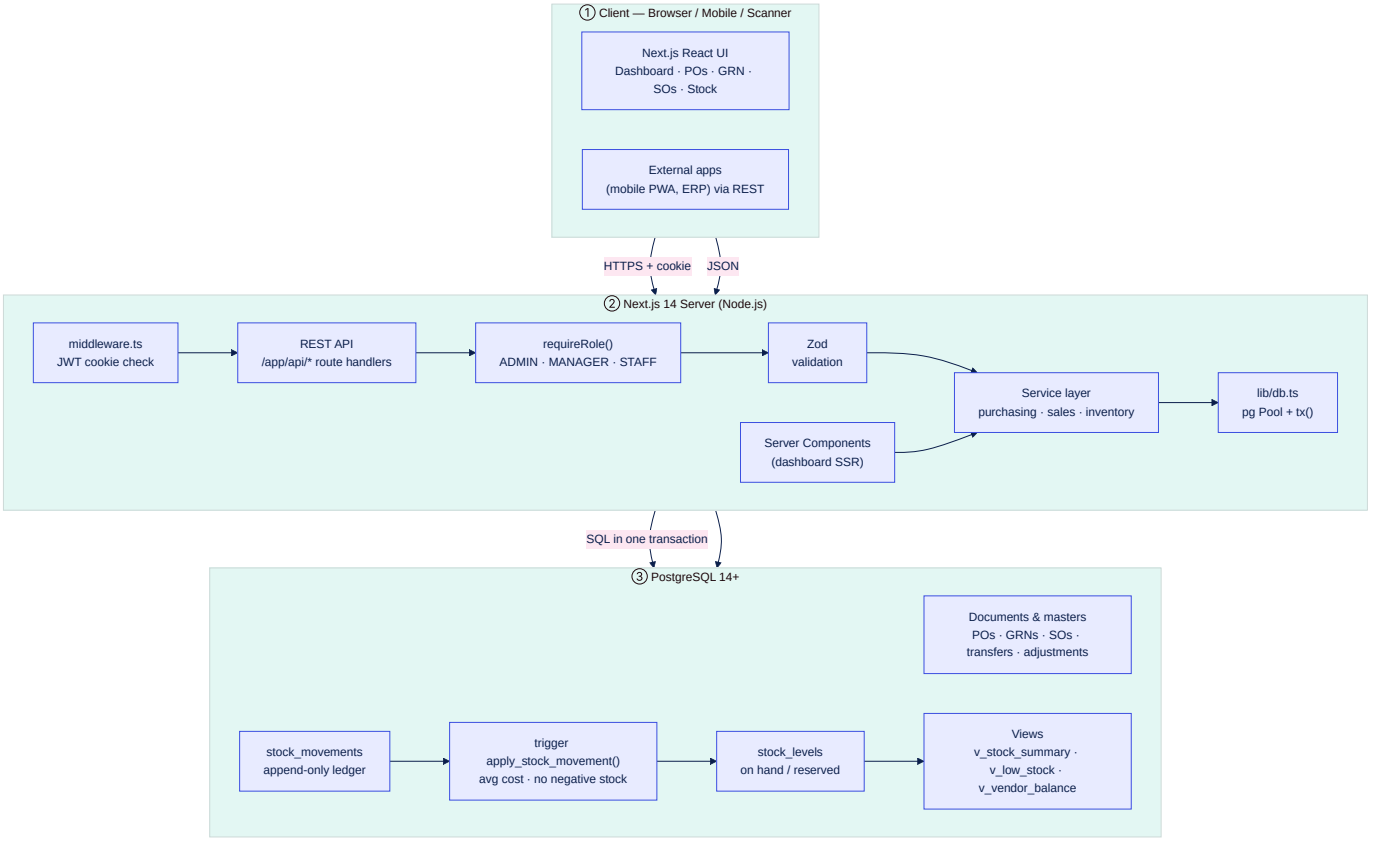
StockPilot is a multi-warehouse inventory system that covers the full cycle: buying from vendors, receiving goods into a warehouse, holding and moving stock, and shipping to customers. Every quantity change goes through one append-only **stock ledger**, so the stock on screen always matches its history and can be audited.

Module	What it does
Dashboard	Stock value, low-stock count, POs awaiting approval, open POs, orders to dispatch, vendor payables, reorder alerts with the preferred vendor, value by warehouse, latest movements
Product master	SKU, category, UOM, barcode, HSN, cost and sale price, GST %, reorder level and reorder quantity
Vendor management	Vendor master (GSTIN, contact, payment terms, rating), vendor price list and lead times, preferred vendor, PO history, payments, outstanding balance
Warehouse management	Multiple warehouses with bin locations, stock by warehouse, transfers between warehouses, adjustments with reason codes
Procurement (inbound)	Purchase orders → approval → goods receipt notes (GRN) with partial receipts, rejected quantities and vendor invoice number
Sales / dispatch (outbound)	Sales orders → confirm (reserves stock) → dispatch (deducts stock) → printable challan
Stock ledger	Complete movement history with running balance, cost, user and document reference
Users & security	JWT session cookie, three roles, permissions checked on every API call

2. Technology Stack

Layer	Choice	Why
Frontend	Next.js 14 App Router, React 18, TypeScript	One codebase for UI and API; server components for fast dashboards
Backend	Next.js Route Handlers (REST) + service layer	Plain JSON API that a mobile app, barcode scanner or ERP can also call
Validation	Zod	Every request body is type-checked before it reaches the database
Database	PostgreSQL 14+ (tested on 16)	ACID transactions, row locks, triggers, CHECK constraints, views
DB driver	`pg` (node-postgres) with pooled connections	Direct, readable SQL; no ORM magic in stock logic
Auth	`jose` JWT (HS256) in httpOnly cookie, `bcryptjs`	Works in Edge middleware and Node routes
Hosting	Any Node 18+ server / Docker / Vercel + managed Postgres (Neon, RDS, Supabase)	No vendor lock-in

3. System Architecture



Request lifecycle

1. **middleware.ts** verifies the JWT cookie on every route. If the user is not signed in, pages redirect to `/login` and APIs return 401.
2. The **route handler** is wrapped by `api()`, which turns errors into clean JSON (400 validation, 403 role, 409 duplicate, 422 business rule).
3. `requireRole()` checks permissions and a **Zod schema** validates the body.
4. The **service layer** runs the business operation inside `tx()` (BEGIN … COMMIT/ROLLBACK) and locks documents with `SELECT … FOR UPDATE`.
5. Stock changes are written *only* as rows in `stock_movements`. The PostgreSQL trigger updates `stock_levels` in the same transaction, so stock can never be half-posted.

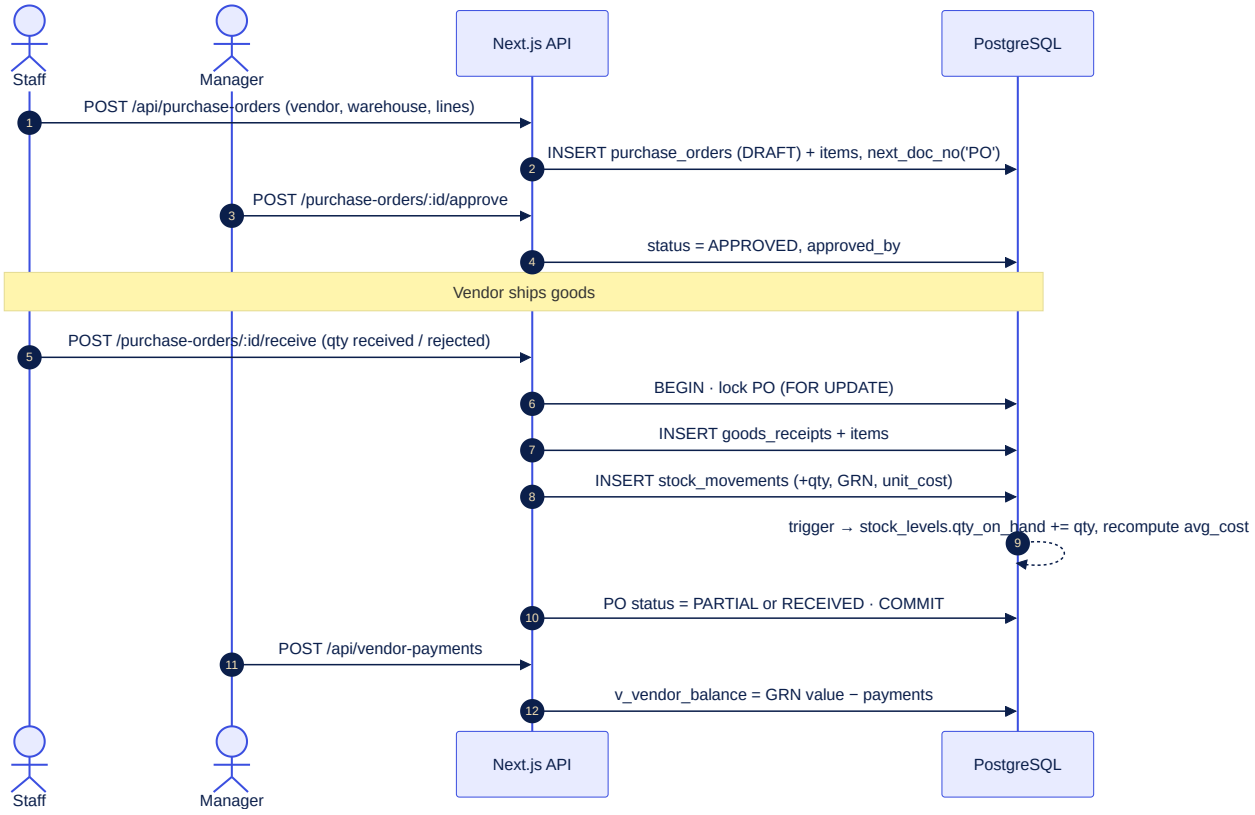
4. Roles & Permissions

Action	ADMIN	MANAGER	STAFF
View dashboard, stock, ledger, documents	✔	✔	✔
Create PO (draft), post GRN	✔	✔	✔
Approve / cancel PO	✔	✔	—
Create / confirm / dispatch sales orders	✔	✔	✔
Stock transfers	✔	✔	✔
Stock adjustments (change stock value)	✔	✔	—
Products, categories, vendors, price lists, vendor payments	✔	✔	—
Warehouses	✔	—	—
Users	✔	—	—

Demo logins (password `Admin@123`): admin@wms.local, manager@wms.local, staff@wms.local. Change these before going live.

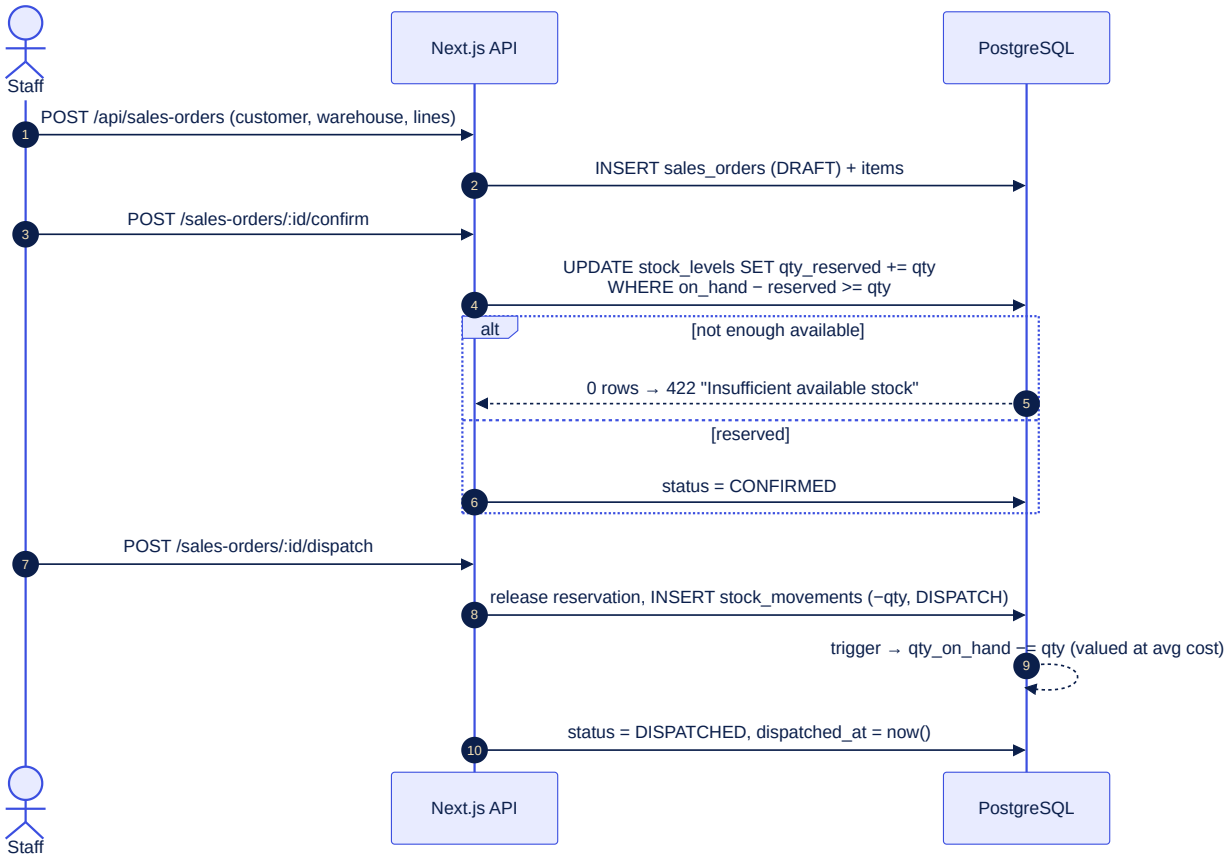
5. Business Process Flows

5.1 Procure-to-Stock (Vendor → Warehouse)



The GRN guards against over-receiving (the quantity cannot exceed what is still pending on the PO). Rejected quantities are recorded on the GRN but are not added to stock. The vendor's payable is the tax-inclusive value of goods received minus payments made.

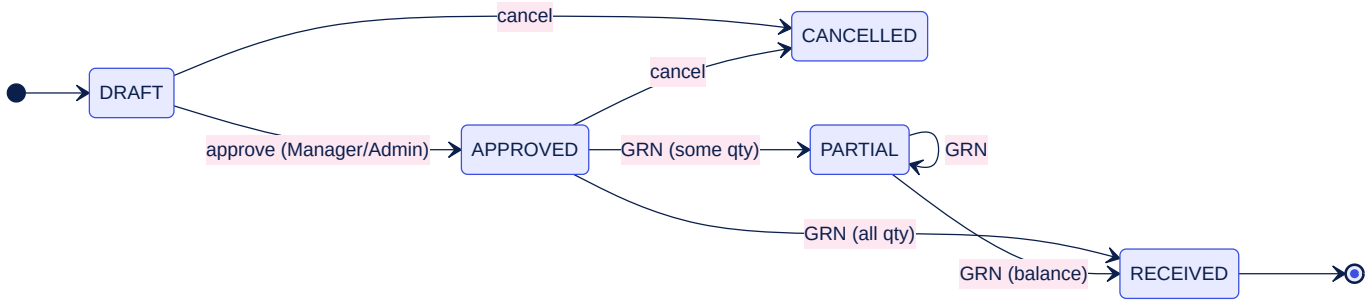
5.2 Order-to-Dispatch (Warehouse → Customer)



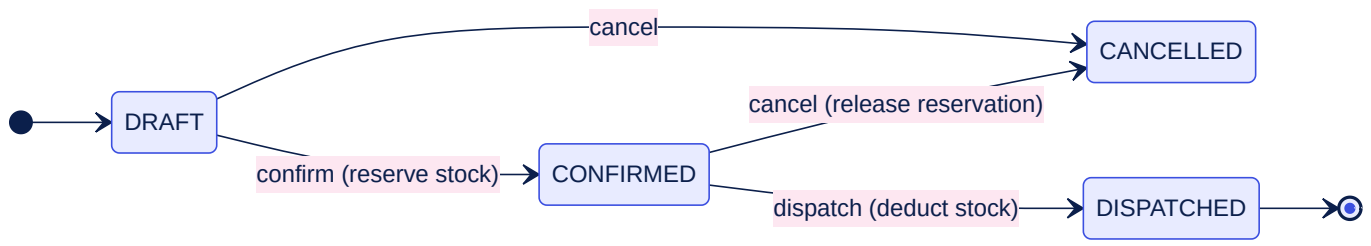
Available = On hand − Reserved. Confirming an order reserves stock so two orders cannot promise the same units. Dispatch releases the reservation and posts the outbound movement in one transaction.

5.3 Document State Machines

Purchase order



Sales order



5.4 Internal Movements

Flow	Steps	Ledger entries
Stock transfer	Choose source & destination warehouse → add lines → post	`TRANSFER_OUT −qty` at source (valued at source average cost) and `TRANSFER_IN +qty` at destination at the same cost, in one transaction
Adjustment	Warehouse + reason (CYCLE_COUNT, DAMAGE, EXPIRY, FOUND, OPENING, OTHER) → signed quantities → post (Manager/Admin)	`ADJUSTMENT ±qty`; positive lines use the entered cost or the product cost price
Reorder	`v_low_stock` lists items where available ≤ reorder level, with the preferred/cheapest vendor → raise a PO for the reorder qty	—
Vendor payment	Vendor page → amount, mode (BANK/UPI/CHEQUE/CASH), UTR reference	— (updates `v_vendor_balance`)

5.5 End-to-end flow summary

Vendor & product master → Purchase order (DRAFT) → Approve → GRN → stock IN →
Store / transfer / adjust → Sales order → Confirm → reserve → Dispatch → stock OUT → Pay vendor

6. Inventory Engine

6.1 Single source of truth: the ledger

The application never runs `UPDATE stock_levels SET qty_on_hand = …` directly. Every in/out is an `INSERT` into `stock_movements`. A `BEFORE INSERT` trigger, `apply_stock_movement()`, then:

1. creates the `stock_levels` row if missing and locks it (`FOR UPDATE`), so two users posting together are handled one after the other;
2. works out the new on-hand quantity and **raises an error if it would go below zero**, which rolls back the whole document;
3. recalculates the **weighted average cost** on inflows: `(old_qty × old_cost + in_qty × in_cost) ÷ new_qty`;
4. values outflows at the current average cost and stamps `balance_after` on the ledger row.

A second trigger makes the ledger **append-only**: updates and deletes are rejected, so mistakes are corrected with a reversing adjustment, which keeps a full audit trail.

6.2 Safety rules enforced in the database

- `CHECK (qty_on_hand >= 0)`, `CHECK (qty_reserved <= qty_on_hand)`
- `CHECK (qty_received <= qty_ordered)` on PO lines
- `CHECK (from_warehouse_id <> to_warehouse_id)` on transfers
- Unique SKU, barcode, vendor code and document numbers; foreign keys on every reference
- Document numbers come from `next_doc_no()` inside the same transaction, so a failed posting doesn't use up a number

6.3 Reporting views

- `v_stock_summary`: on hand, reserved, available, average cost and value per product × warehouse
- `v_low_stock`: items at or below reorder level, with the suggested vendor
- `v_vendor_balance`: goods received, amount paid, balance due and open POs per vendor

7. Database Design

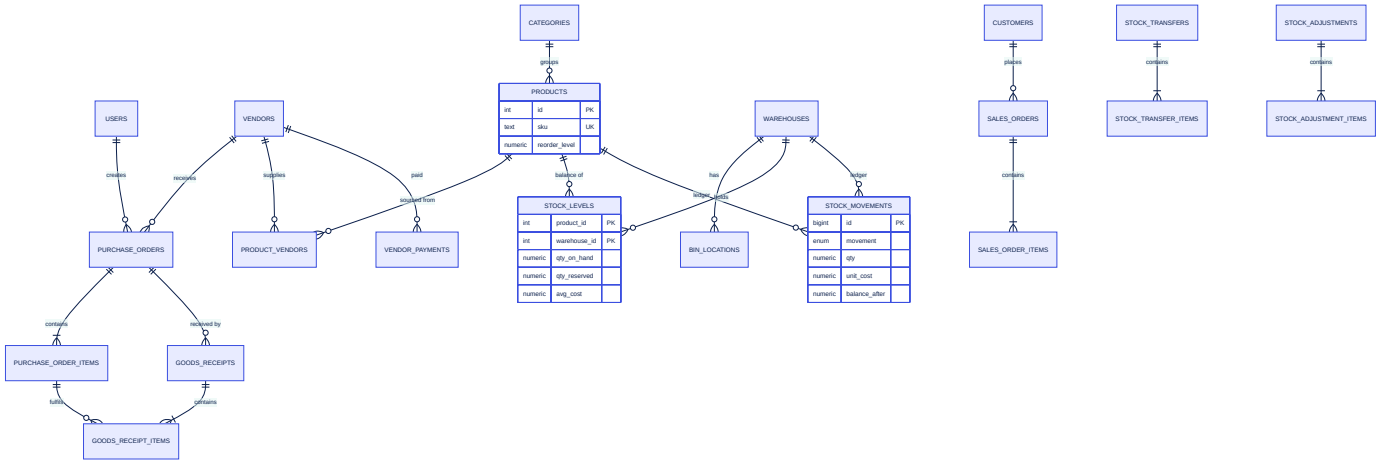


Table catalogue

Table	Purpose
`users`	Login accounts with role ADMIN / MANAGER / STAFF; bcrypt password hash
`warehouses, bin_locations`	Physical sites and aisle-rack-shelf bins
`categories, products`	Item master: SKU, UOM, barcode, HSN, cost/sale price, GST %, reorder level & qty
`vendors, product_vendors`	Supplier master (GSTIN, payment terms, rating) and per-vendor price list with lead time / preferred flag
`customers`	Consignees for outbound orders
`stock_levels`	Current balance per product × warehouse: on hand, reserved, weighted average cost. Written only by trigger and reservation updates
`stock_movements`	Append-only ledger of every in/out (OPENING, GRN, DISPATCH, TRANSFER_IN/OUT, ADJUSTMENT, VENDOR_RETURN) with balance_after
`purchase_orders, purchase_order_items`	Procurement documents with ordered vs received quantities
`goods_receipts, goods_receipt_items`	GRN header/lines: accepted and rejected quantity, unit cost, vendor invoice no.
`vendor_payments`	Payments against vendor (optionally a PO) — drives payables
`sales_orders, sales_order_items`	Outbound orders: DRAFT → CONFIRMED (reserved) → DISPATCHED
`stock_transfers (+items)`	Moves between warehouses, cost carried from source
`stock_adjustments (+items)`	Signed corrections with reason code
`doc_sequences + next_doc_no()`	Gap-safe, transactional numbering: PO-2026-00001, GRN-..., SO-..., TRF-..., ADJ-...

The complete DDL, triggers, functions and views are in Appendix A.

8. REST API Reference

All endpoints accept and return JSON. Auth uses the `wms_session` httpOnly cookie set by `/api/auth/login`. Errors return `{"error": "message"}` with status 400 / 401 / 403 / 404 / 409 / 422.

Method	Endpoint	Role	Description
POST	`/api/auth/login`	Public	Sign in; sets httpOnly JWT cookie (12h)
POST	`/api/auth/logout`	Any	Clear session
GET	`/api/auth/me`	Any	Current user
GET/POST	`/api/products`	Read: all · Write: Admin/Manager	List (?q=) / create product
GET/PUT	`/api/products/:id`	Admin/Manager	Get / update product
GET/POST	`/api/categories`	Admin/Manager	Product categories
GET/POST	`/api/vendors`	Admin/Manager	List / create vendor
GET/PUT	`/api/vendors/:id`	Admin/Manager	Vendor + balance due, open POs
GET/POST	`/api/vendors/:id/products`	Admin/Manager	Vendor price list (upsert link)
GET/POST	`/api/vendor-payments`	Admin/Manager	Payments (?vendor_id=)
GET/POST	`/api/warehouses`	Admin	Warehouses
GET/POST	`/api/customers`	All	Customers
GET/POST	`/api/purchase-orders`	All	List (?status=) / create DRAFT PO
GET	`/api/purchase-orders/:id`	All	PO with lines and GRNs
POST	`/api/purchase-orders/:id/approve`	Admin/Manager	DRAFT → APPROVED
POST	`/api/purchase-orders/:id/cancel`	Admin/Manager	DRAFT/APPROVED → CANCELLED
POST	`/api/purchase-orders/:id/receive`	All	Post GRN → stock in, PARTIAL/RECEIVED
GET/POST	`/api/sales-orders`	All	List / create DRAFT SO
GET	`/api/sales-orders/:id`	All	SO with live availability
POST	`/api/sales-orders/:id/confirm`	All	Reserve stock → CONFIRMED
POST	`/api/sales-orders/:id/dispatch`	All	Deduct stock → DISPATCHED
POST	`/api/sales-orders/:id/cancel`	All	Cancel + release reservation
GET/POST	`/api/transfers`	All	Inter-warehouse transfer (out + in at avg cost)
GET/POST	`/api/adjustments`	Write: Admin/Manager	Cycle count, damage, expiry…
GET	`/api/stock`	All	Stock levels (?warehouse_id=&q=)
GET	`/api/stock/movements`	All	Stock ledger (?product_id=)
GET	`/api/dashboard`	All	KPIs, low stock, value by warehouse
GET	`/api/lookups`	All	Dropdown data for forms

GET/POST	`/api/users`	Admin	User management
PUT	`/api/users/:id`	Admin	Update user / reset password

Example: create a purchase order

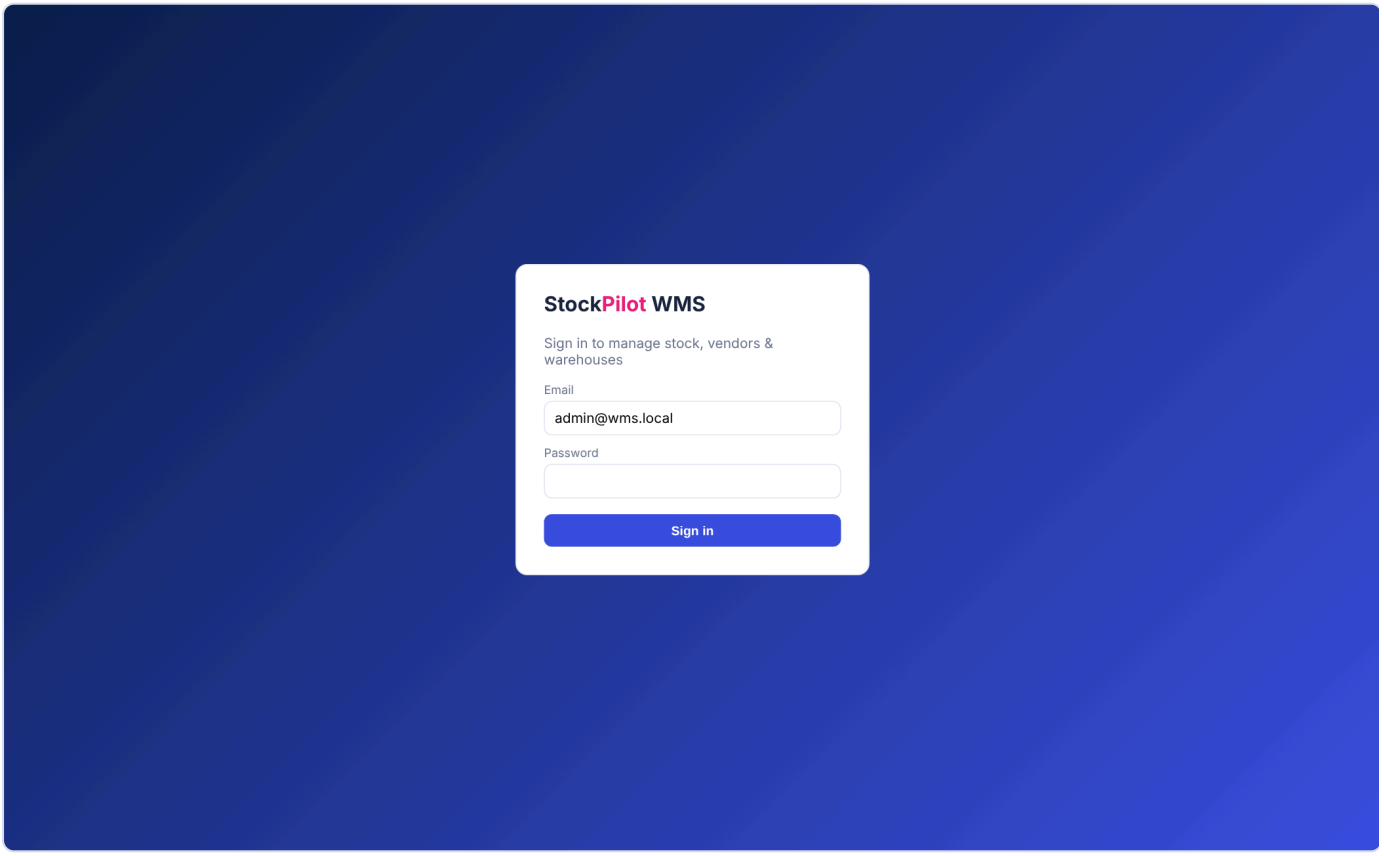
```
POST /api/purchase-orders
{ "vendor_id": 1, "warehouse_id": 1, "expected_date": "2026-11-10",
  "items": [ { "product_id": 5, "qty": 500, "unit_price": 42, "tax_rate": 18 } ] }

POST /api/purchase-orders/2/receive
{ "invoice_no": "INV-9", "items": [ { "po_item_id": 3, "qty_received": 300, "qty_rejected": 5 } ] }
```

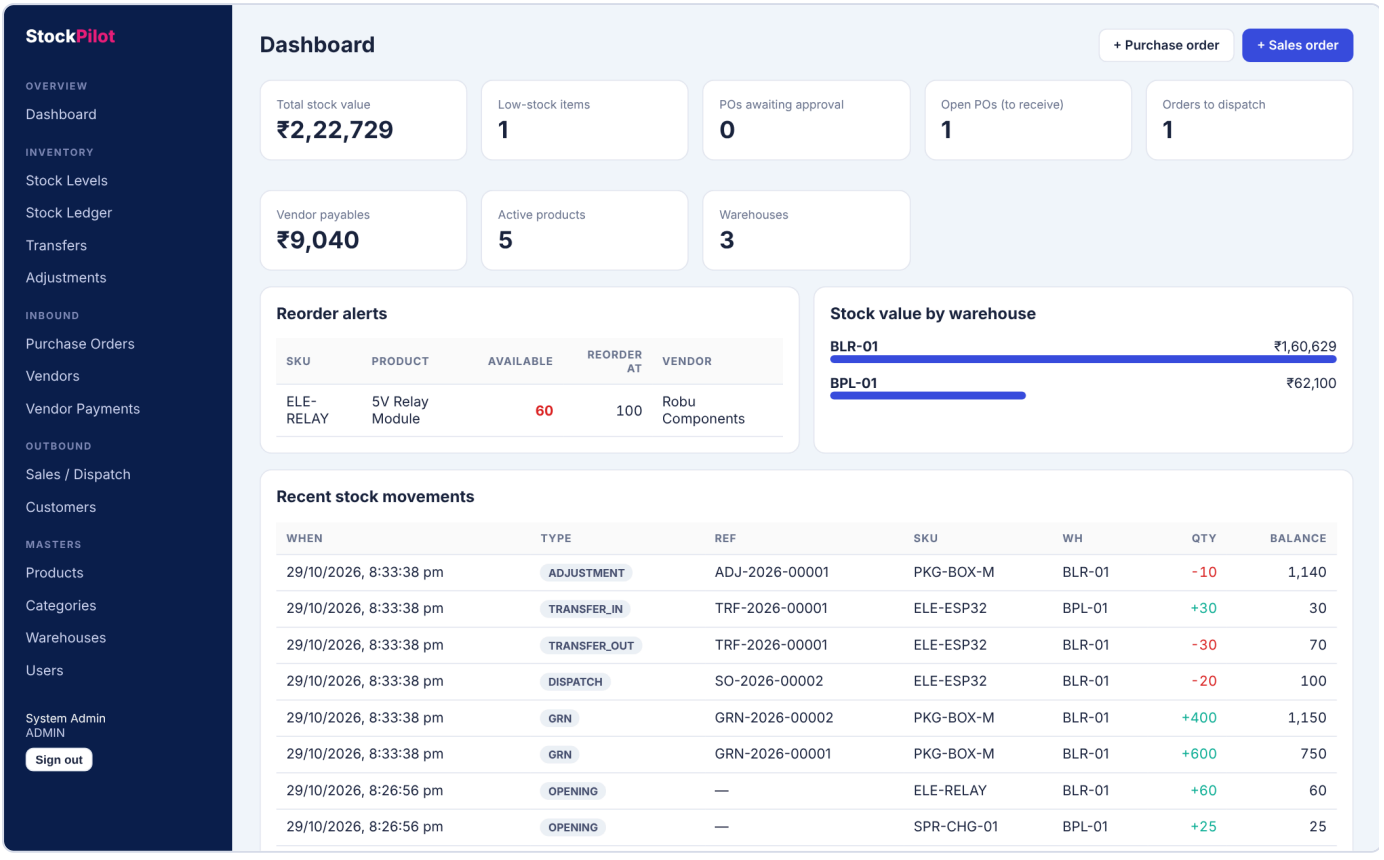
9. Project Structure

```
./.env.example
./.gitignore
./app/(app)/adjustments/page.tsx
./app/(app)/categories/page.tsx
./app/(app)/customers/page.tsx
./app/(app)/layout.tsx
./app/(app)/page.tsx
./app/(app)/products/page.tsx
./app/(app)/purchase-orders/[id]/page.tsx
./app/(app)/purchase-orders/new/page.tsx
./app/(app)/purchase-orders/page.tsx
./app/(app)/sales-orders/[id]/page.tsx
./app/(app)/sales-orders/new/page.tsx
./app/(app)/sales-orders/page.tsx
./app/(app)/stock/movements/page.tsx
./app/(app)/stock/page.tsx
./app/(app)/transfers/page.tsx
./app/(app)/users/page.tsx
./app/(app)/vendor-payments/page.tsx
./app/(app)/vendors/[id]/page.tsx
./app/(app)/vendors/page.tsx
./app/(app)/warehouses/page.tsx
./app/api/adjustments/route.ts
./app/api/auth/login/route.ts
./app/api/auth/logout/route.ts
./app/api/auth/me/route.ts
./app/api/categories/route.ts
./app/api/customers/[id]/route.ts
./app/api/customers/route.ts
./app/api/dashboard/route.ts
./app/api/lookups/route.ts
./app/api/products/[id]/route.ts
./app/api/products/route.ts
./app/api/purchase-orders/[id]/approve/route.ts
./app/api/purchase-orders/[id]/cancel/route.ts
./app/api/purchase-orders/[id]/receive/route.ts
./app/api/purchase-orders/[id]/route.ts
./app/api/purchase-orders/route.ts
./app/api/sales-orders/[id]/cancel/route.ts
./app/api/sales-orders/[id]/confirm/route.ts
./app/api/sales-orders/[id]/dispatch/route.ts
./app/api/sales-orders/[id]/route.ts
./app/api/sales-orders/route.ts
./app/api/stock/movements/route.ts
./app/api/stock/route.ts
./app/api/transfers/route.ts
./app/api/users/[id]/route.ts
./app/api/users/route.ts
./app/api/vendor-payments/route.ts
./app/api/vendors/[id]/products/route.ts
./app/api/vendors/[id]/route.ts
./app/api/vendors/route.ts
./app/api/warehouses/[id]/route.ts
./app/api/warehouses/route.ts
./app/globals.css
./app/layout.tsx
./app/login/page.tsx
./components/LineItems.tsx
./components/LogoutButton.tsx
./components/MasterPage.tsx
./components/Status.tsx
./db/schema.sql
./db/seed.sql
./lib/api.ts
./lib/auth.ts
./lib/client.ts
./lib/crud.ts
./lib/db.ts
./lib/services/inventory.ts
./lib/services/purchasing.ts
./lib/services/sales.ts
./lib/session.ts
./lib/validators.ts
./middleware.ts
./next.config.mjs
./package-lock.json
./package.json
./tsconfig.json
```

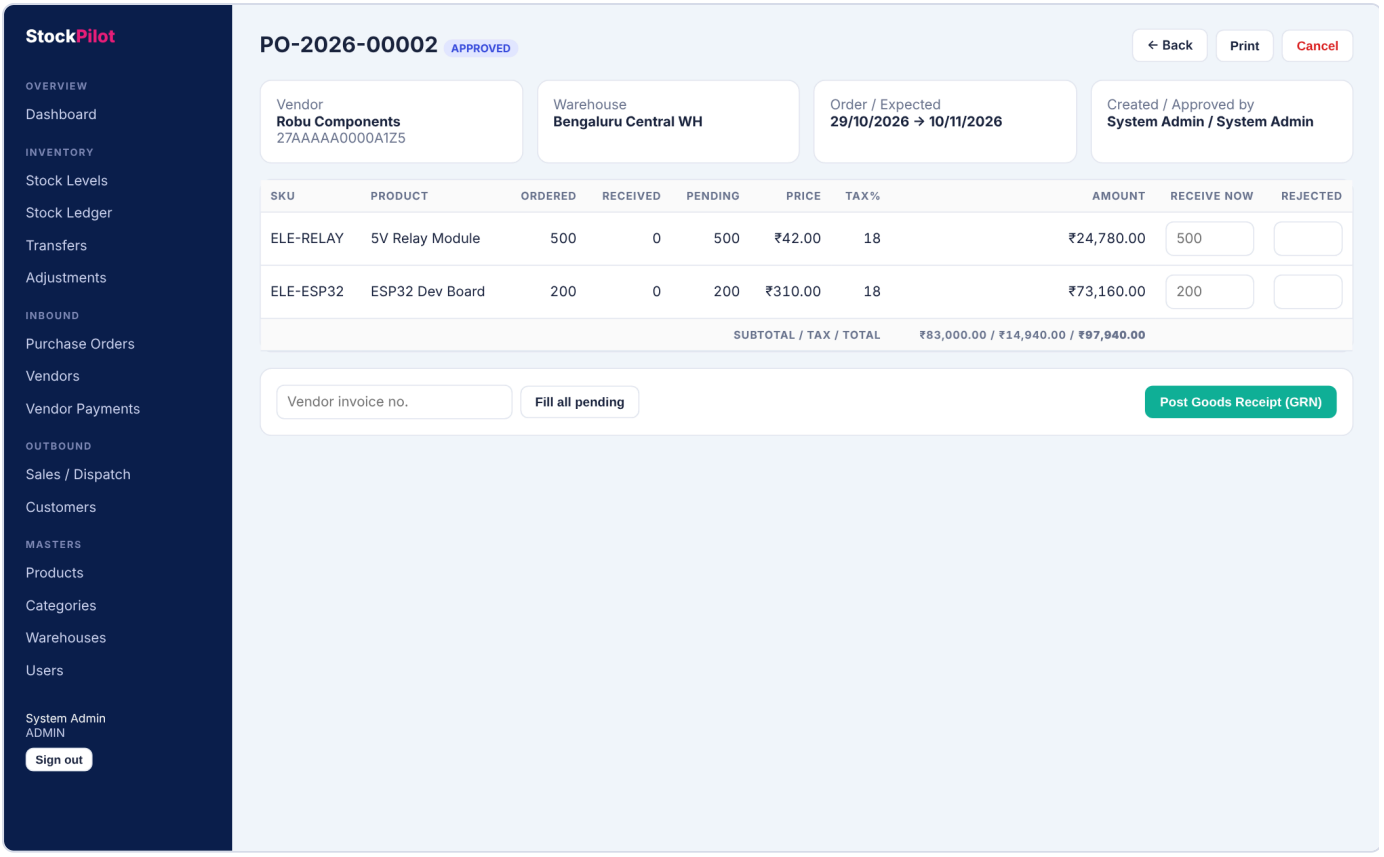
10. Application Screens



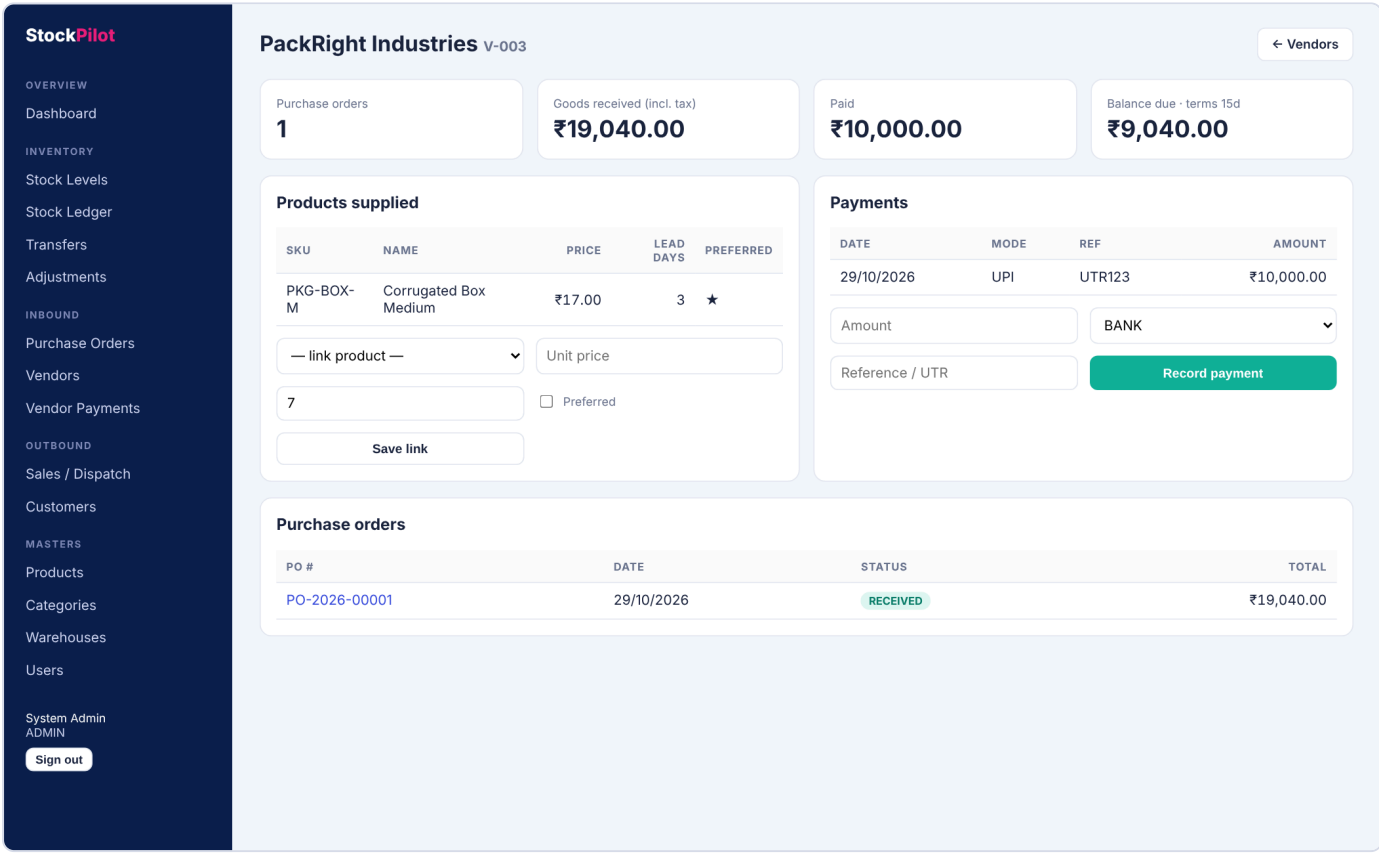
Login screen



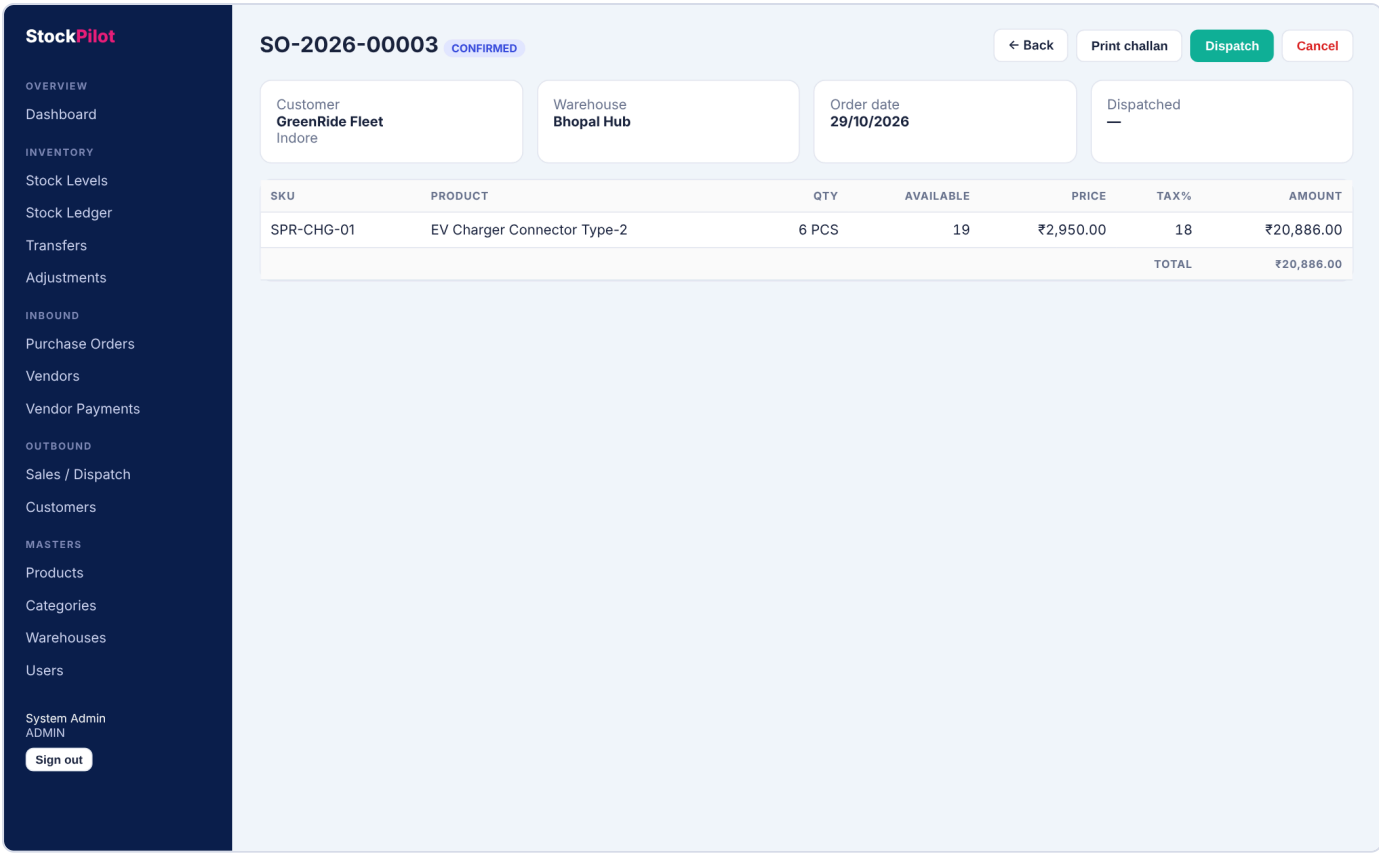
Dashboard — KPIs, reorder alerts, stock value by warehouse, recent movements



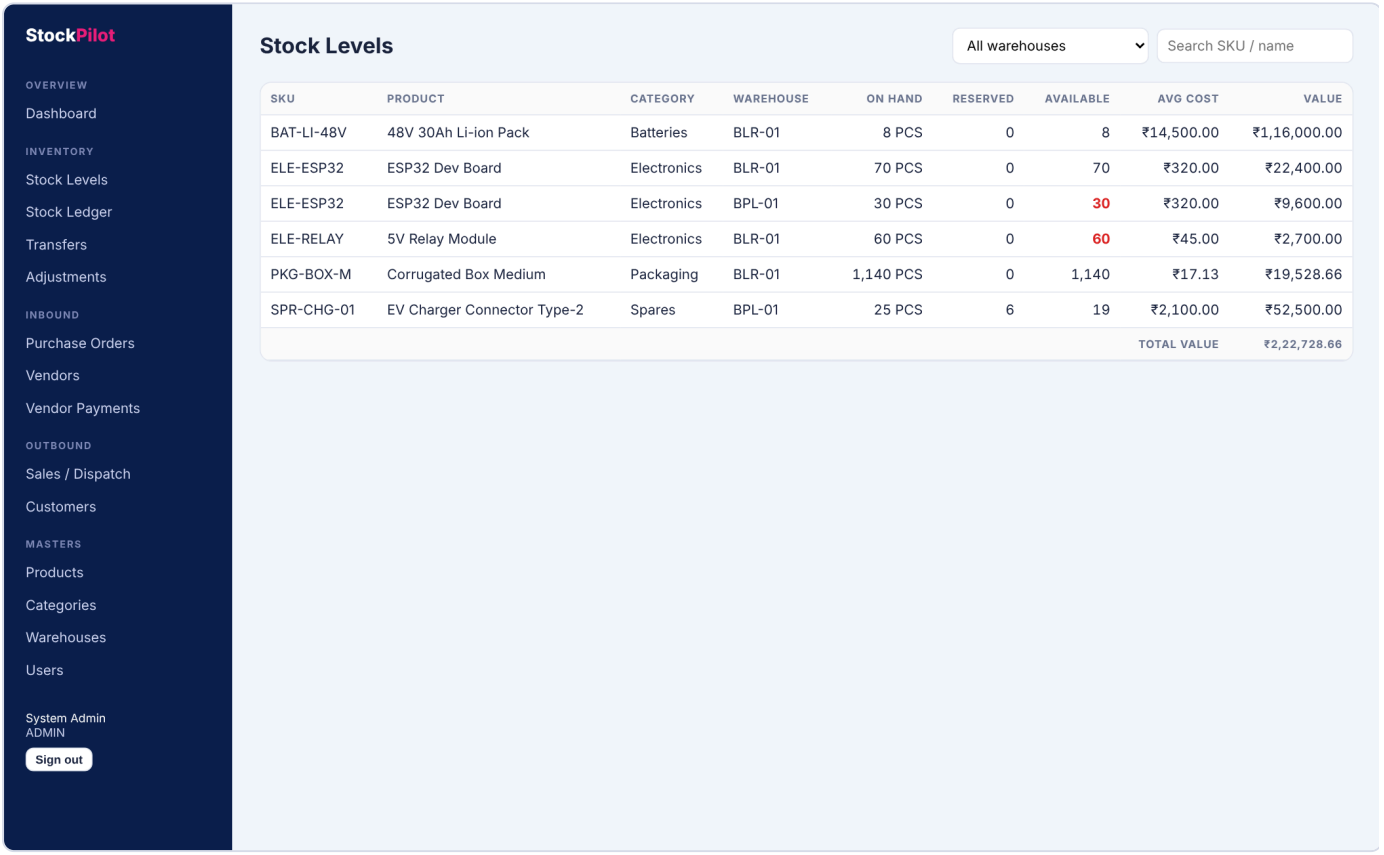
Purchase order (APPROVED) with goods-receipt entry



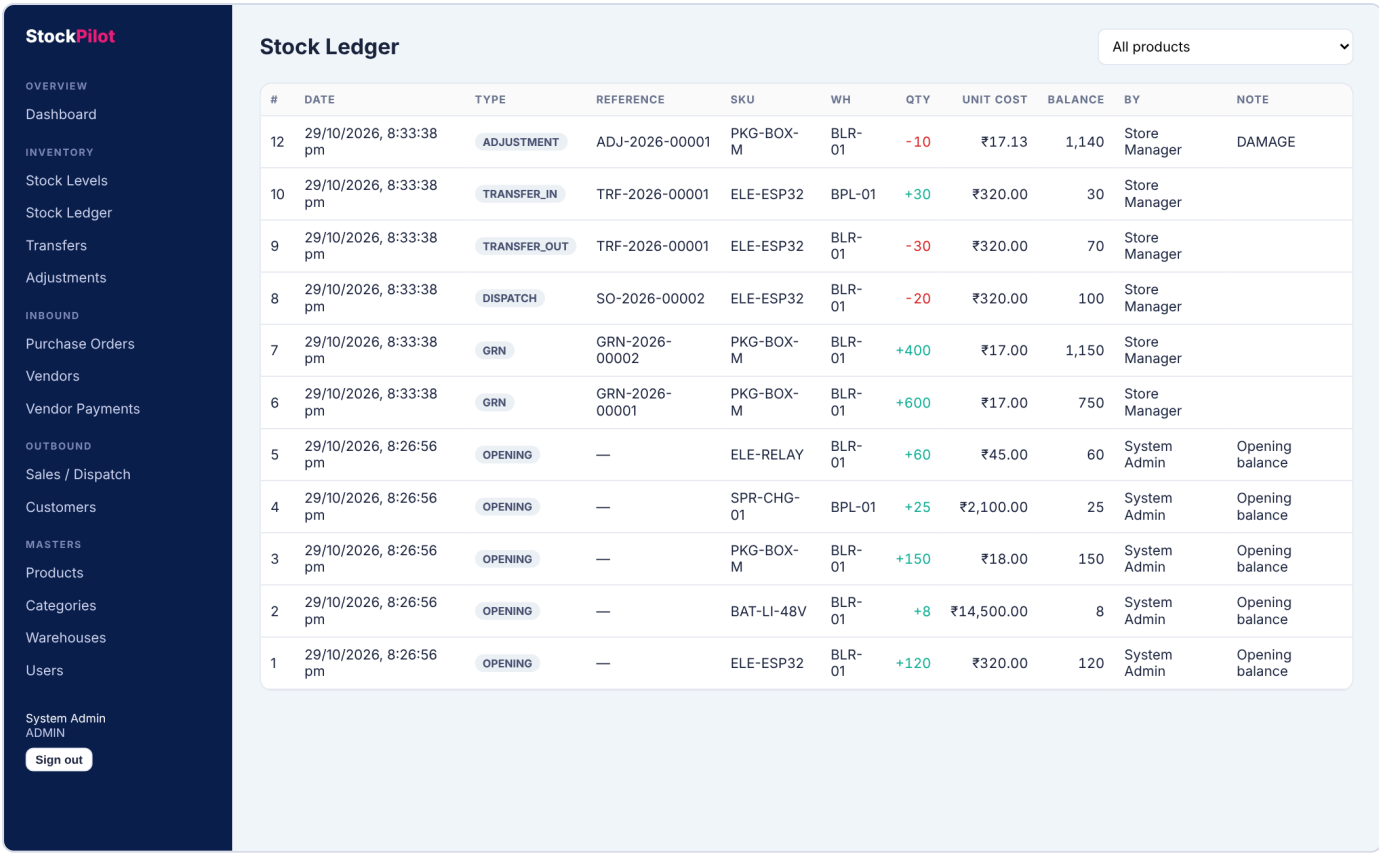
Vendor profile — price list, payments, balance due, PO history



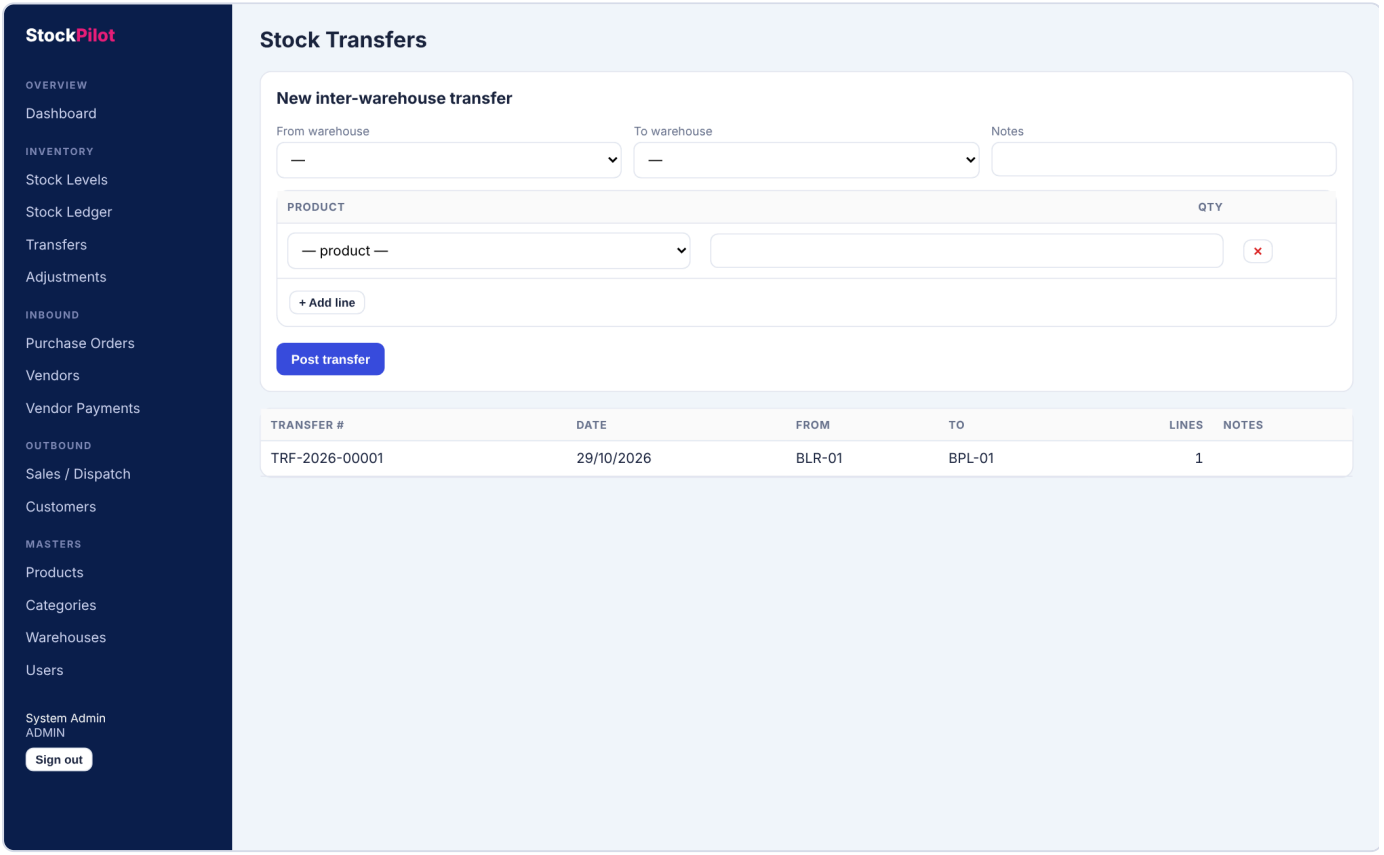
Sales order confirmed — stock reserved, ready to dispatch



Stock levels by warehouse with valuation



Stock ledger — every movement with running balance



Inter-warehouse transfer screen

11. Setup & Deployment

Local development

```
# 1. Prerequisites: Node.js 18+, PostgreSQL 14+
createdb wms

# 2. Configure
cp .env.example .env
#   DATABASE_URL=postgres://postgres:postgres@localhost:5432/wms
#   JWT_SECRET=<long random string, 32+ chars>

# 3. Install, create schema + demo data
npm install
export $(cat .env | xargs) && npm run db:init

# 4. Run
npm run dev          # http://localhost:3000   (admin@wms.local / Admin@123)

# Production
npm run build && npm start
```

Production checklist

- Set a strong `JWT_SECRET`; serve over HTTPS (the cookie is marked `secure` in production).
- Delete or change the demo users and passwords in `db/seed.sql`; seed only masters you need.
- Use a managed Postgres with daily backups and point-in-time recovery; use a separate DB role with only DML rights for the app.
- Run behind Nginx or a load balancer; the app is stateless, so you can run several instances.
- Docker: `FROM node:20-alpine` → `npm ci && npm run build` → `CMD ["npm","start"]`.

Suggested next phases

- Batch / serial / expiry tracking (add `batch_no`, `expiry_date` to movements) and FEFO picking
- Bin-level stock (add `bin_id` to `stock_levels`) with put-away and pick lists
- Barcode / QR scanning (mobile PWA) calling the same REST API
- Vendor returns (the `VENDOR_RETURN` movement type is already in the enum), purchase invoices and 3-way match
- Email PO to vendor as PDF, GST e-way bill integration, approval limits by amount
- Reports: ageing, ABC analysis, vendor on-time delivery, stock valuation by date

Appendix A — PostgreSQL Schema & Seed

db/schema.sql

```
-- ================================================================
--  StockPilot WMS — Stock, Vendor & Warehouse Management
--  PostgreSQL 14+ schema
-- ================================================================
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- ---------- Enums ----------------------------------------------
CREATE TYPE user_role       AS ENUM ('ADMIN','MANAGER','STAFF');
CREATE TYPE po_status       AS ENUM ('DRAFT','APPROVED','PARTIAL','RECEIVED','CANCELLED');
CREATE TYPE so_status       AS ENUM ('DRAFT','CONFIRMED','DISPATCHED','CANCELLED');
CREATE TYPE movement_type   AS ENUM ('GRN','DISPATCH','TRANSFER_OUT','TRANSFER_IN',
                                     'ADJUSTMENT','VENDOR_RETURN','OPENING');

-- ---------- Users ----------------------------------------------
CREATE TABLE users (
  id            SERIAL PRIMARY KEY,
  name          TEXT NOT NULL,
  email         TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  role          user_role NOT NULL DEFAULT 'STAFF',
  active        BOOLEAN NOT NULL DEFAULT TRUE,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ---------- Masters --------------------------------------------
CREATE TABLE warehouses (
  id         SERIAL PRIMARY KEY,
  code       TEXT NOT NULL UNIQUE,
  name       TEXT NOT NULL,
  address    TEXT,
  city       TEXT,
  manager    TEXT,
  active     BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE bin_locations (
  id           SERIAL PRIMARY KEY,
  warehouse_id INT NOT NULL REFERENCES warehouses(id) ON DELETE CASCADE,
  code         TEXT NOT NULL,              -- e.g. A-01-03 (aisle-rack-shelf)
  description  TEXT,
  UNIQUE (warehouse_id, code)
);

CREATE TABLE categories (
  id   SERIAL PRIMARY KEY,
  name TEXT NOT NULL UNIQUE
);

CREATE TABLE products (
  id            SERIAL PRIMARY KEY,
  sku           TEXT NOT NULL UNIQUE,
  name          TEXT NOT NULL,
  category_id   INT REFERENCES categories(id),
  uom           TEXT NOT NULL DEFAULT 'PCS',
  barcode       TEXT UNIQUE,
  hsn_code      TEXT,
  cost_price    NUMERIC(14,2) NOT NULL DEFAULT 0,
  sale_price    NUMERIC(14,2) NOT NULL DEFAULT 0,
  tax_rate      NUMERIC(5,2)  NOT NULL DEFAULT 18,
  reorder_level NUMERIC(14,3) NOT NULL DEFAULT 0,
  reorder_qty   NUMERIC(14,3) NOT NULL DEFAULT 0,
  active        BOOLEAN NOT NULL DEFAULT TRUE,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE vendors (
  id                SERIAL PRIMARY KEY,
  code              TEXT NOT NULL UNIQUE,
```

```
  name               TEXT NOT NULL,
  contact_person     TEXT,
  email              TEXT,
  phone              TEXT,
  gstin              TEXT,
  address            TEXT,
  payment_terms_days INT NOT NULL DEFAULT 30,
  rating             NUMERIC(2,1) CHECK (rating BETWEEN 0 AND 5),
  active             BOOLEAN NOT NULL DEFAULT TRUE,
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- which vendor supplies which product, at what price / lead time
CREATE TABLE product_vendors (
  product_id     INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  vendor_id      INT NOT NULL REFERENCES vendors(id)  ON DELETE CASCADE,
  vendor_sku     TEXT,
  unit_price     NUMERIC(14,2) NOT NULL DEFAULT 0,
  lead_time_days INT NOT NULL DEFAULT 7,
  preferred      BOOLEAN NOT NULL DEFAULT FALSE,
  PRIMARY KEY (product_id, vendor_id)
);

CREATE TABLE customers (
  id         SERIAL PRIMARY KEY,
  name       TEXT NOT NULL,
  email      TEXT,
  phone      TEXT,
  gstin      TEXT,
  address    TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ---------- Document numbering ------------------------------------
CREATE TABLE doc_sequences (
  prefix  TEXT PRIMARY KEY,
  last_no INT NOT NULL DEFAULT 0
);

CREATE OR REPLACE FUNCTION next_doc_no(p_prefix TEXT) RETURNS TEXT AS $$
DECLARE n INT;
BEGIN
  INSERT INTO doc_sequences(prefix, last_no) VALUES (p_prefix, 1)
  ON CONFLICT (prefix) DO UPDATE SET last_no = doc_sequences.last_no + 1
  RETURNING last_no INTO n;
  RETURN p_prefix || '-' || to_char(now(),'YYYY') || '-' || lpad(n::text, 5, '0');
END $$ LANGUAGE plpgsql;

-- ---------- Inventory core ----------------------------------------
-- Current balance per product per warehouse (maintained ONLY by trigger)
CREATE TABLE stock_levels (
  product_id   INT NOT NULL REFERENCES products(id),
  warehouse_id INT NOT NULL REFERENCES warehouses(id),
  qty_on_hand  NUMERIC(14,3) NOT NULL DEFAULT 0,
  qty_reserved NUMERIC(14,3) NOT NULL DEFAULT 0,
  avg_cost     NUMERIC(14,4) NOT NULL DEFAULT 0,
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (product_id, warehouse_id),
  CHECK (qty_on_hand  >= 0),
  CHECK (qty_reserved >= 0),
  CHECK (qty_reserved <= qty_on_hand)
);

-- Immutable stock ledger: every in/out is a row here
CREATE TABLE stock_movements (
  id            BIGSERIAL PRIMARY KEY,
  product_id    INT NOT NULL REFERENCES products(id),
  warehouse_id  INT NOT NULL REFERENCES warehouses(id),
  movement      movement_type NOT NULL,
  qty           NUMERIC(14,3) NOT NULL CHECK (qty <> 0),  -- +in / -out
  unit_cost     NUMERIC(14,4) NOT NULL DEFAULT 0,
  ref_type      TEXT,          -- 'PO','GRN','SO','TRF','ADJ'
  ref_id        INT,
  ref_no        TEXT,
  note          TEXT,
  balance_after NUMERIC(14,3),
```

```
  created_by    INT REFERENCES users(id),
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_mov_product ON stock_movements(product_id, warehouse_id, created_at DESC);
CREATE INDEX idx_mov_ref     ON stock_movements(ref_type, ref_id);

-- Trigger: apply each movement to stock_levels (weighted-average costing)
CREATE OR REPLACE FUNCTION apply_stock_movement() RETURNS TRIGGER AS $$
DECLARE
  cur stock_levels%ROWTYPE;
  new_qty NUMERIC(14,3);
BEGIN
  INSERT INTO stock_levels(product_id, warehouse_id)
  VALUES (NEW.product_id, NEW.warehouse_id)
  ON CONFLICT DO NOTHING;

  SELECT * INTO cur FROM stock_levels
   WHERE product_id = NEW.product_id AND warehouse_id = NEW.warehouse_id
   FOR UPDATE;

  new_qty := cur.qty_on_hand + NEW.qty;
  IF new_qty < 0 THEN
    RAISE EXCEPTION 'Insufficient stock for product % in warehouse % (on hand %, requested %)',
      NEW.product_id, NEW.warehouse_id, cur.qty_on_hand, -NEW.qty
      USING ERRCODE = 'check_violation';
  END IF;

  UPDATE stock_levels SET
    qty_on_hand = new_qty,
    avg_cost = CASE
      WHEN NEW.qty > 0 AND new_qty > 0 THEN
        ((cur.qty_on_hand * cur.avg_cost) + (NEW.qty * NEW.unit_cost)) / new_qty
      ELSE cur.avg_cost END,
    updated_at = now()
  WHERE product_id = NEW.product_id AND warehouse_id = NEW.warehouse_id;

  NEW.balance_after := new_qty;
  IF NEW.qty < 0 AND NEW.unit_cost = 0 THEN
    NEW.unit_cost := cur.avg_cost;    -- outflows valued at average cost
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_apply_stock_movement
BEFORE INSERT ON stock_movements
FOR EACH ROW EXECUTE FUNCTION apply_stock_movement();

-- Ledger is append-only
CREATE OR REPLACE FUNCTION forbid_change() RETURNS TRIGGER AS $$
BEGIN RAISE EXCEPTION 'stock_movements is append-only; post a reversing entry instead'; END $$
LANGUAGE plpgsql;
CREATE TRIGGER trg_mov_no_update BEFORE UPDATE OR DELETE ON stock_movements
FOR EACH ROW EXECUTE FUNCTION forbid_change();

-- ---------- Procurement (Inbound) ----------------------------------
CREATE TABLE purchase_orders (
  id            SERIAL PRIMARY KEY,
  po_number     TEXT NOT NULL UNIQUE,
  vendor_id     INT NOT NULL REFERENCES vendors(id),
  warehouse_id  INT NOT NULL REFERENCES warehouses(id),
  status        po_status NOT NULL DEFAULT 'DRAFT',
  order_date    DATE NOT NULL DEFAULT CURRENT_DATE,
  expected_date DATE,
  notes         TEXT,
  subtotal      NUMERIC(14,2) NOT NULL DEFAULT 0,
  tax_total     NUMERIC(14,2) NOT NULL DEFAULT 0,
  total         NUMERIC(14,2) NOT NULL DEFAULT 0,
  created_by    INT REFERENCES users(id),
  approved_by   INT REFERENCES users(id),
  approved_at   TIMESTAMPTZ,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE purchase_order_items (
  id           SERIAL PRIMARY KEY,
  po_id        INT NOT NULL REFERENCES purchase_orders(id) ON DELETE CASCADE,
```

```
  product_id   INT NOT NULL REFERENCES products(id),
  qty_ordered  NUMERIC(14,3) NOT NULL CHECK (qty_ordered > 0),
  qty_received NUMERIC(14,3) NOT NULL DEFAULT 0,
  unit_price   NUMERIC(14,2) NOT NULL DEFAULT 0,
  tax_rate     NUMERIC(5,2)  NOT NULL DEFAULT 0,
  CHECK (qty_received <= qty_ordered)
);

CREATE TABLE goods_receipts (
  id            SERIAL PRIMARY KEY,
  grn_number    TEXT NOT NULL UNIQUE,
  po_id         INT NOT NULL REFERENCES purchase_orders(id),
  warehouse_id  INT NOT NULL REFERENCES warehouses(id),
  received_date DATE NOT NULL DEFAULT CURRENT_DATE,
  invoice_no    TEXT,
  notes         TEXT,
  total_value   NUMERIC(14,2) NOT NULL DEFAULT 0,
  created_by    INT REFERENCES users(id),
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE goods_receipt_items (
  id           SERIAL PRIMARY KEY,
  grn_id       INT NOT NULL REFERENCES goods_receipts(id) ON DELETE CASCADE,
  po_item_id   INT NOT NULL REFERENCES purchase_order_items(id),
  product_id   INT NOT NULL REFERENCES products(id),
  qty_received NUMERIC(14,3) NOT NULL CHECK (qty_received >= 0),
  qty_rejected NUMERIC(14,3) NOT NULL DEFAULT 0 CHECK (qty_rejected >= 0),
  unit_cost    NUMERIC(14,2) NOT NULL DEFAULT 0
);

CREATE TABLE vendor_payments (
  id         SERIAL PRIMARY KEY,
  vendor_id  INT NOT NULL REFERENCES vendors(id),
  po_id      INT REFERENCES purchase_orders(id),
  amount     NUMERIC(14,2) NOT NULL CHECK (amount > 0),
  paid_on    DATE NOT NULL DEFAULT CURRENT_DATE,
  mode       TEXT NOT NULL DEFAULT 'BANK',   -- BANK / UPI / CHEQUE / CASH
  reference  TEXT,
  created_by INT REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ---------- Sales / Dispatch (Outbound) ----------------------------
CREATE TABLE sales_orders (
  id            SERIAL PRIMARY KEY,
  so_number     TEXT NOT NULL UNIQUE,
  customer_id   INT NOT NULL REFERENCES customers(id),
  warehouse_id  INT NOT NULL REFERENCES warehouses(id),
  status        so_status NOT NULL DEFAULT 'DRAFT',
  order_date    DATE NOT NULL DEFAULT CURRENT_DATE,
  notes         TEXT,
  subtotal      NUMERIC(14,2) NOT NULL DEFAULT 0,
  tax_total     NUMERIC(14,2) NOT NULL DEFAULT 0,
  total         NUMERIC(14,2) NOT NULL DEFAULT 0,
  dispatched_at TIMESTAMPTZ,
  created_by    INT REFERENCES users(id),
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE sales_order_items (
  id         SERIAL PRIMARY KEY,
  so_id      INT NOT NULL REFERENCES sales_orders(id) ON DELETE CASCADE,
  product_id INT NOT NULL REFERENCES products(id),
  qty        NUMERIC(14,3) NOT NULL CHECK (qty > 0),
  unit_price NUMERIC(14,2) NOT NULL DEFAULT 0,
  tax_rate   NUMERIC(5,2)  NOT NULL DEFAULT 0
);

-- ---------- Internal movements -------------------------------------
CREATE TABLE stock_transfers (
  id                SERIAL PRIMARY KEY,
  transfer_number   TEXT NOT NULL UNIQUE,
  from_warehouse_id INT NOT NULL REFERENCES warehouses(id),
  to_warehouse_id   INT NOT NULL REFERENCES warehouses(id),
  notes             TEXT,
```

```
  created_by        INT REFERENCES users(id),
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (from_warehouse_id <> to_warehouse_id)
);
CREATE TABLE stock_transfer_items (
  id          SERIAL PRIMARY KEY,
  transfer_id INT NOT NULL REFERENCES stock_transfers(id) ON DELETE CASCADE,
  product_id  INT NOT NULL REFERENCES products(id),
  qty         NUMERIC(14,3) NOT NULL CHECK (qty > 0)
);

CREATE TABLE stock_adjustments (
  id           SERIAL PRIMARY KEY,
  adj_number   TEXT NOT NULL UNIQUE,
  warehouse_id INT NOT NULL REFERENCES warehouses(id),
  reason       TEXT NOT NULL,          -- DAMAGE / CYCLE_COUNT / EXPIRY / FOUND / OPENING
  notes        TEXT,
  created_by   INT REFERENCES users(id),
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE stock_adjustment_items (
  id            SERIAL PRIMARY KEY,
  adjustment_id INT NOT NULL REFERENCES stock_adjustments(id) ON DELETE CASCADE,
  product_id    INT NOT NULL REFERENCES products(id),
  qty_change    NUMERIC(14,3) NOT NULL CHECK (qty_change <> 0),
  unit_cost     NUMERIC(14,2) NOT NULL DEFAULT 0
);

-- ---------- Reporting views ----------------------------------------
CREATE VIEW v_stock_summary AS
SELECT p.id AS product_id, p.sku, p.name, c.name AS category, p.uom,
       w.id AS warehouse_id, w.code AS warehouse_code, w.name AS warehouse,
       s.qty_on_hand, s.qty_reserved,
       (s.qty_on_hand - s.qty_reserved) AS qty_available,
       s.avg_cost, ROUND(s.qty_on_hand * s.avg_cost, 2) AS stock_value,
       p.reorder_level
FROM stock_levels s
JOIN products p   ON p.id = s.product_id
JOIN warehouses w ON w.id = s.warehouse_id
LEFT JOIN categories c ON c.id = p.category_id;

CREATE VIEW v_low_stock AS
SELECT p.id AS product_id, p.sku, p.name, p.reorder_level, p.reorder_qty,
       COALESCE(SUM(s.qty_on_hand - s.qty_reserved),0) AS qty_available,
       (SELECT v.name FROM product_vendors pv JOIN vendors v ON v.id = pv.vendor_id
         WHERE pv.product_id = p.id ORDER BY pv.preferred DESC, pv.unit_price LIMIT 1) AS preferred_vendor
FROM products p
LEFT JOIN stock_levels s ON s.product_id = p.id
WHERE p.active
GROUP BY p.id
HAVING COALESCE(SUM(s.qty_on_hand - s.qty_reserved),0) <= p.reorder_level;

CREATE VIEW v_vendor_balance AS
SELECT v.id AS vendor_id, v.code, v.name,
       COALESCE((SELECT SUM(g.total_value) FROM goods_receipts g
                  JOIN purchase_orders po ON po.id = g.po_id
                 WHERE po.vendor_id = v.id),0) AS total_received_value,
       COALESCE((SELECT SUM(amount) FROM vendor_payments vp WHERE vp.vendor_id = v.id),0) AS total_paid,
       COALESCE((SELECT SUM(g.total_value) FROM goods_receipts g
                  JOIN purchase_orders po ON po.id = g.po_id
                 WHERE po.vendor_id = v.id),0)
     - COALESCE((SELECT SUM(amount) FROM vendor_payments vp WHERE vp.vendor_id = v.id),0) AS balance_due,
       (SELECT COUNT(*) FROM purchase_orders po WHERE po.vendor_id = v.id
          AND po.status IN ('APPROVED','PARTIAL')) AS open_pos
FROM vendors v;
```

db/seed.sql

```
-- Demo data. Default password for all users: Admin@123
INSERT INTO users(name,email,password_hash,role) VALUES
 ('System Admin','admin@wms.local',  crypt('Admin@123', gen_salt('bf',10)),'ADMIN'),
 ('Store Manager','manager@wms.local',crypt('Admin@123', gen_salt('bf',10)),'MANAGER'),
 ('Store Keeper','staff@wms.local',   crypt('Admin@123', gen_salt('bf',10)),'STAFF');
```

```
INSERT INTO warehouses(code,name,address,city,manager) VALUES
 ('BLR-01','Bengaluru Central WH','Manyata Tech Park','Bengaluru','Ravi'),
 ('BPL-01','Bhopal Hub','Indrapuri Sector C','Bhopal','Pranjal'),
 ('DEL-01','Delhi DC','Okhla Phase II','Delhi','Amit');

INSERT INTO bin_locations(warehouse_id,code,description) VALUES
 (1,'A-01-01','Fast movers'),(1,'B-02-01','Bulk'),(2,'A-01-01','General');

INSERT INTO categories(name) VALUES ('Electronics'),('Batteries'),('Packaging'),('Spares');

INSERT INTO products(sku,name,category_id,uom,cost_price,sale_price,tax_rate,reorder_level,reorder_qty) VALUES
 ('ELE-ESP32','ESP32 Dev Board',1,'PCS',320,450,18,50,200),
 ('BAT-LI-48V','48V 30Ah Li-ion Pack',2,'PCS',14500,18900,18,5,20),
 ('PKG-BOX-M','Corrugated Box Medium',3,'PCS',18,30,12,200,1000),
 ('SPR-CHG-01','EV Charger Connector Type-2',4,'PCS',2100,2950,18,10,40),
 ('ELE-RELAY','5V Relay Module',1,'PCS',45,80,18,100,500);

INSERT INTO vendors(code,name,contact_person,email,phone,gstin,address,payment_terms_days,rating) VALUES
 ('V-001','Robu Components','Sanjay','sales@robu.example','9800000001','27AAAAA0000A1Z5','Pune',30,4.5),
 ('V-002','PowerCell
Energy','Meena','orders@powercell.example','9800000002','29BBBBB1111B1Z2','Bengaluru',45,4.0),
 ('V-003','PackRight
Industries','Imran','hello@packright.example','9800000003','23CCCCC2222C1Z9','Indore',15,3.8);

INSERT INTO product_vendors(product_id,vendor_id,vendor_sku,unit_price,lead_time_days,preferred) VALUES
 (1,1,'R-ESP32',310,5,true),(5,1,'R-RLY5',42,5,true),
 (2,2,'PC-48-30',14200,14,true),(4,2,'PC-T2',2050,10,true),
 (3,3,'PR-BM',17,3,true);

INSERT INTO customers(name,email,phone,address) VALUES
 ('NIVASHOP Online','orders@nivashop.example','9811111111','Satna'),
 ('GreenRide Fleet','ops@greenride.example','9822222222','Indore');

-- Opening stock via the ledger (trigger fills stock_levels)
INSERT INTO stock_movements(product_id,warehouse_id,movement,qty,unit_cost,ref_type,note,created_by) VALUES
 (1,1,'OPENING',120,320,'OPEN','Opening balance',1),
 (2,1,'OPENING',8,14500,'OPEN','Opening balance',1),
 (3,1,'OPENING',150,18,'OPEN','Opening balance',1),
 (4,2,'OPENING',25,2100,'OPEN','Opening balance',1),
 (5,1,'OPENING',60,45,'OPEN','Opening balance',1);
```

Appendix B — Complete Next.js Source Code

package.json

```
{
  "name": "stockpilot-wms",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "db:init": "psql \"$DATABASE_URL\" -v ON_ERROR_STOP=1 -f db/schema.sql && psql \"$DATABASE_URL\" -v
ON_ERROR_STOP=1 -f db/seed.sql"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "jose": "^5.9.6",
    "next": "14.2.15",
    "pg": "^8.13.0",
    "react": "18.3.1",
    "react-dom": "18.3.1",
    "zod": "^3.23.8"
  },
  "devDependencies": {
    "@types/bcryptjs": "^2.4.6",
    "@types/node": "^20.16.0",
    "@types/pg": "^8.11.10",
    "@types/react": "^18.3.11",
    "@types/react-dom": "^18.3.0",
    "typescript": "^5.6.3"
  }
}
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020", "lib": ["dom","dom.iterable","esnext"], "allowJs": false,
    "skipLibCheck": true, "strict": true, "noEmit": true, "esModuleInterop": true,
    "module": "esnext", "moduleResolution": "bundler", "resolveJsonModule": true,
    "isolatedModules": true, "jsx": "preserve", "incremental": true,
    "plugins": [{ "name": "next" }], "paths": { "@/*": ["./*"] }
  },
  "include": ["next-env.d.ts","**/*.ts","**/*.tsx",".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

next.config.mjs

```
/** @type {import('next').NextConfig} */
const nextConfig = { experimental: { serverComponentsExternalPackages: ['pg', 'bcryptjs'] } };
export default nextConfig;
```

.env.example

```
DATABASE_URL=postgres://postgres:postgres@localhost:5432/wms
JWT_SECRET=change-this-to-a-long-random-string-min-32-chars
```

middleware.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { COOKIE, verifySession } from '@/lib/session';

export async function middleware(req: NextRequest) {
  const { pathname } = req.nextUrl;
  if (pathname.startsWith('/login') || pathname.startsWith('/api/auth/login')) return NextResponse.next();
```

```
  const user = await verifySession(req.cookies.get(COOKIE)?.value);
  if (!user) {
    if (pathname.startsWith('/api/')) return NextResponse.json({ error: 'Not authenticated' }, { status: 401 });
    return NextResponse.redirect(new URL('/login', req.url));
  }
  return NextResponse.next();
}
export const config = { matcher: ['/((?!_next/static|_next/image|favicon.ico).*)'] };
```

lib/db.ts

```
import { Pool, PoolClient, QueryResultRow, types } from 'pg';

// Return DATE columns as 'YYYY-MM-DD' strings (avoid timezone shifts)
types.setTypeParser(1082, v => v);

const g = globalThis as unknown as { __pgPool?: Pool };
export const pool =
  g.__pgPool ?? new Pool({ connectionString: process.env.DATABASE_URL, max: 10 });
if (process.env.NODE_ENV !== 'production') g.__pgPool = pool;

export async function query<T extends QueryResultRow = any>(sql: string, params: unknown[] = []) {
  const res = await pool.query<T>(sql, params);
  return res.rows;
}

export async function one<T extends QueryResultRow = any>(sql: string, params: unknown[] = []) {
  const rows = await query<T>(sql, params);
  return rows[0] ?? null;
}

/** Run a function inside a single DB transaction (BEGIN/COMMIT/ROLLBACK). */
export async function tx<T>(fn: (c: PoolClient) => Promise<T>): Promise<T> {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    const out = await fn(client);
    await client.query('COMMIT');
    return out;
  } catch (e) {
    await client.query('ROLLBACK');
    throw e;
  } finally {
    client.release();
  }
}
```

lib/session.ts

```
// Edge-safe JWT helpers (used by middleware and server code)
import { SignJWT, jwtVerify } from 'jose';

export type Role = 'ADMIN' | 'MANAGER' | 'STAFF';
export type SessionUser = { id: number; name: string; email: string; role: Role };
export const COOKIE = 'wms_session';

const secret = () =>
  new TextEncoder().encode(process.env.JWT_SECRET || 'dev-only-secret-change-me-please-32ch');

export async function signSession(u: SessionUser) {
  return new SignJWT({ ...u })
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuedAt()
    .setExpirationTime('12h')
    .sign(secret());
}

export async function verifySession(token?: string): Promise<SessionUser | null> {
  if (!token) return null;
  try {
    const { payload } = await jwtVerify(token, secret());
    return payload as unknown as SessionUser;
  } catch {
```

```
    return null;
  }
}
```

lib/auth.ts

```
import { cookies } from 'next/headers';
import { redirect } from 'next/navigation';
import { COOKIE, Role, SessionUser, verifySession } from './session';

export async function getUser(): Promise<SessionUser | null> {
  return verifySession(cookies().get(COOKIE)?.value);
}

/** For server pages: redirect to /login when not signed in */
export async function requirePageUser(): Promise<SessionUser> {
  const u = await getUser();
  if (!u) redirect('/login');
  return u;
}

export class HttpError extends Error {
  constructor(public status: number, message: string) { super(message); }
}

/** For API routes: throw 401/403 */
export async function requireRole(...roles: Role[]): Promise<SessionUser> {
  const u = await getUser();
  if (!u) throw new HttpError(401, 'Not authenticated');
  if (roles.length && !roles.includes(u.role)) throw new HttpError(403, 'Insufficient permission');
  return u;
}
```

lib/api.ts

```
import { NextResponse } from 'next/server';
import { ZodError } from 'zod';
import { HttpError } from './auth';

type Handler = (req: Request, ctx: { params: Record<string, string> }) => Promise<unknown>;

/** Wraps a route handler: JSON response + uniform error mapping */
export function api(fn: Handler) {
  return async (req: Request, ctx: { params: Record<string, string> }) => {
    try {
      const data = await fn(req, ctx);
      return NextResponse.json(data ?? { ok: true });
    } catch (e: any) {
      if (e instanceof HttpError) return NextResponse.json({ error: e.message }, { status: e.status });
      if (e instanceof ZodError)
        return NextResponse.json({ error: e.issues.map(i => `${i.path.join('.')}: ${i.message}`).join('; ') }, {
status: 400 });
      if (e?.code === '23505') return NextResponse.json({ error: 'Duplicate value: ' + (e.detail ?? '') }, {
status: 409 });
      if (e?.code === '23514') return NextResponse.json({ error: e.message }, { status: 422 });
      if (e?.code === '23503') return NextResponse.json({ error: 'Referenced record missing or in use' }, {
status: 422 });
      console.error(e);
      return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
    }
  };
}
```

lib/validators.ts

```
import { z } from 'zod';

const num = z.coerce.number();
const optStr = z.string().trim().optional().nullable().transform(v => (v ? v : null));

export const productSchema = z.object({
  sku: z.string().trim().min(1), name: z.string().trim().min(1),
```

```
  category_id: z.coerce.number().int().optional().nullable(),
  uom: z.string().trim().default('PCS'), barcode: optStr, hsn_code: optStr,
  cost_price: num.min(0).default(0), sale_price: num.min(0).default(0),
  tax_rate: num.min(0).max(100).default(18),
  reorder_level: num.min(0).default(0), reorder_qty: num.min(0).default(0),
  active: z.coerce.boolean().default(true),
});

export const vendorSchema = z.object({
  code: z.string().trim().min(1), name: z.string().trim().min(1),
  contact_person: optStr, email: optStr, phone: optStr, gstin: optStr, address: optStr,
  payment_terms_days: z.coerce.number().int().min(0).default(30),
  rating: z.coerce.number().min(0).max(5).optional().nullable(),
  active: z.coerce.boolean().default(true),
});

export const warehouseSchema = z.object({
  code: z.string().trim().min(1), name: z.string().trim().min(1),
  address: optStr, city: optStr, manager: optStr, active: z.coerce.boolean().default(true),
});

export const customerSchema = z.object({
  name: z.string().trim().min(1), email: optStr, phone: optStr, gstin: optStr, address: optStr,
});

export const categorySchema = z.object({ name: z.string().trim().min(1) });

export const userSchema = z.object({
  name: z.string().trim().min(1), email: z.string().email(),
  password: z.string().min(6).optional(), role: z.enum(['ADMIN', 'MANAGER', 'STAFF']),
  active: z.coerce.boolean().default(true),
});

const line = z.object({
  product_id: z.coerce.number().int().positive(),
  qty: num.positive(),
  unit_price: num.min(0).default(0),
  tax_rate: num.min(0).max(100).default(0),
});

export const poSchema = z.object({
  vendor_id: z.coerce.number().int().positive(),
  warehouse_id: z.coerce.number().int().positive(),
  expected_date: optStr, notes: optStr,
  items: z.array(line).min(1, 'Add at least one item'),
});

export const grnSchema = z.object({
  invoice_no: optStr, notes: optStr,
  items: z.array(z.object({
    po_item_id: z.coerce.number().int().positive(),
    qty_received: num.min(0), qty_rejected: num.min(0).default(0),
  })).min(1),
});

export const soSchema = z.object({
  customer_id: z.coerce.number().int().positive(),
  warehouse_id: z.coerce.number().int().positive(),
  notes: optStr, items: z.array(line).min(1, 'Add at least one item'),
});

export const transferSchema = z.object({
  from_warehouse_id: z.coerce.number().int().positive(),
  to_warehouse_id: z.coerce.number().int().positive(),
  notes: optStr,
  items: z.array(z.object({ product_id: z.coerce.number().int().positive(), qty: num.positive() })).min(1),
}).refine(d => d.from_warehouse_id !== d.to_warehouse_id, 'Source and destination must differ');

export const adjustmentSchema = z.object({
  warehouse_id: z.coerce.number().int().positive(),
  reason: z.enum(['CYCLE_COUNT', 'DAMAGE', 'EXPIRY', 'FOUND', 'OPENING', 'OTHER']),
  notes: optStr,
  items: z.array(z.object({
    product_id: z.coerce.number().int().positive(),
    qty_change: num.refine(v => v !== 0, 'Quantity change cannot be 0'),
    unit_cost: num.min(0).default(0),
```

```
  })).min(1),
});

export const paymentSchema = z.object({
  vendor_id: z.coerce.number().int().positive(),
  po_id: z.coerce.number().int().positive().optional().nullable(),
  amount: num.positive(), paid_on: optStr,
  mode: z.enum(['BANK', 'UPI', 'CHEQUE', 'CASH']).default('BANK'), reference: optStr,
});
```

lib/crud.ts

```
import { query, one } from './db';

/** Generic master-data CRUD — table/columns are fixed server-side (never from user input). */
export function crud(table: string, columns: string[], orderBy = 'id DESC') {
  return {
    list: (search?: string) =>
      search
        ? query(`SELECT * FROM ${table} WHERE ${columns.filter(c => ['name', 'code', 'sku', 'email'].includes(c))
            .map(c => `${c} ILIKE $1`).join(' OR ')} ORDER BY ${orderBy} LIMIT 500`, [`%${search}%`])
        : query(`SELECT * FROM ${table} ORDER BY ${orderBy} LIMIT 500`),
    get: (id: number) => one(`SELECT * FROM ${table} WHERE id=$1`, [id]),
    create: (data: Record<string, unknown>) => {
      const cols = columns.filter(c => c in data);
      return one(
        `INSERT INTO ${table}(${cols.join(',')}) VALUES (${cols.map((_, i) => `$${i + 1}`).join(',')}) RETURNING
*`,
        cols.map(c => data[c]));
    },
    update: (id: number, data: Record<string, unknown>) => {
      const cols = columns.filter(c => c in data);
      return one(
        `UPDATE ${table} SET ${cols.map((c, i) => `${c}=$${i + 1}`).join(',')} WHERE id=$${cols.length + 1}
RETURNING *`,
        [...cols.map(c => data[c]), id]);
    },
  };
}

export const products   = crud('products',
['sku','name','category_id','uom','barcode','hsn_code','cost_price','sale_price','tax_rate','reorder_level','reor
der_qty','active']);
export const vendors    = crud('vendors',
['code','name','contact_person','email','phone','gstin','address','payment_terms_days','rating','active']);
export const warehouses = crud('warehouses', ['code','name','address','city','manager','active'], 'code');
export const customers  = crud('customers',  ['name','email','phone','gstin','address']);
export const categories = crud('categories', ['name'], 'name');
```

lib/services/purchasing.ts

```
import { PoolClient } from 'pg';
import { tx, query, one } from '../db';
import { HttpError } from '../auth';

type Line = { product_id: number; qty: number; unit_price: number; tax_rate: number };

function totals(items: Line[]) {
  const subtotal = items.reduce((s, i) => s + i.qty * i.unit_price, 0);
  const tax = items.reduce((s, i) => s + (i.qty * i.unit_price * i.tax_rate) / 100, 0);
  return { subtotal: +subtotal.toFixed(2), tax: +tax.toFixed(2), total: +(subtotal + tax).toFixed(2) };
}

export async function createPO(d: { vendor_id: number; warehouse_id: number; expected_date: string | null; notes:
string | null; items: Line[] }, userId: number) {
  return tx(async c => {
    const t = totals(d.items);
    const { rows: [po] } = await c.query(
      `INSERT INTO
purchase_orders(po_number,vendor_id,warehouse_id,expected_date,notes,subtotal,tax_total,total,created_by)
       VALUES (next_doc_no('PO'),$1,$2,$3,$4,$5,$6,$7,$8) RETURNING *`,
      [d.vendor_id, d.warehouse_id, d.expected_date, d.notes, t.subtotal, t.tax, t.total, userId]);
    for (const i of d.items)
```

```
      await c.query(
        `INSERT INTO purchase_order_items(po_id,product_id,qty_ordered,unit_price,tax_rate) VALUES
($1,$2,$3,$4,$5)`,
        [po.id, i.product_id, i.qty, i.unit_price, i.tax_rate]);
    return po;
  });
}

async function lockPO(c: PoolClient, id: number) {
  const { rows: [po] } = await c.query(`SELECT * FROM purchase_orders WHERE id=$1 FOR UPDATE`, [id]);
  if (!po) throw new HttpError(404, 'Purchase order not found');
  return po;
}

export async function approvePO(id: number, userId: number) {
  return tx(async c => {
    const po = await lockPO(c, id);
    if (po.status !== 'DRAFT') throw new HttpError(422, `Cannot approve a ${po.status} PO`);
    const { rows: [r] } = await c.query(
      `UPDATE purchase_orders SET status='APPROVED', approved_by=$2, approved_at=now() WHERE id=$1 RETURNING *`,
[id, userId]);
    return r;
  });
}

export async function cancelPO(id: number) {
  return tx(async c => {
    const po = await lockPO(c, id);
    if (!['DRAFT', 'APPROVED'].includes(po.status)) throw new HttpError(422, `Cannot cancel a ${po.status} PO`);
    const { rows: [r] } = await c.query(`UPDATE purchase_orders SET status='CANCELLED' WHERE id=$1 RETURNING *`,
[id]);
    return r;
  });
}

/** Goods Receipt: posts GRN, increases stock via ledger, updates PO status */
export async function receivePO(id: number, d: { invoice_no: string | null; notes: string | null;
  items: { po_item_id: number; qty_received: number; qty_rejected: number }[] }, userId: number) {
  return tx(async c => {
    const po = await lockPO(c, id);
    if (!['APPROVED', 'PARTIAL'].includes(po.status)) throw new HttpError(422, `PO must be APPROVED to receive
(is ${po.status})`);
    const accepted = d.items.filter(i => i.qty_received > 0 || i.qty_rejected > 0);
    if (!accepted.length) throw new HttpError(422, 'Enter quantity for at least one line');

    const { rows: [grn] } = await c.query(
      `INSERT INTO goods_receipts(grn_number,po_id,warehouse_id,invoice_no,notes,created_by)
       VALUES (next_doc_no('GRN'),$1,$2,$3,$4,$5) RETURNING *`,
      [id, po.warehouse_id, d.invoice_no, d.notes, userId]);

    let value = 0;
    for (const i of accepted) {
      const { rows: [line] } = await c.query(
        `SELECT * FROM purchase_order_items WHERE id=$1 AND po_id=$2 FOR UPDATE`, [i.po_item_id, id]);
      if (!line) throw new HttpError(422, `Line ${i.po_item_id} not on this PO`);
      const pending = Number(line.qty_ordered) - Number(line.qty_received);
      if (i.qty_received > pending) throw new HttpError(422, `Receiving ${i.qty_received} exceeds pending
${pending}`);

      await c.query(
        `INSERT INTO goods_receipt_items(grn_id,po_item_id,product_id,qty_received,qty_rejected,unit_cost)
         VALUES ($1,$2,$3,$4,$5,$6)`, [grn.id, line.id, line.product_id, i.qty_received, i.qty_rejected,
line.unit_price]);
      if (i.qty_received > 0) {
        await c.query(`UPDATE purchase_order_items SET qty_received = qty_received + $1 WHERE id=$2`,
[i.qty_received, line.id]);
        await c.query(
          `INSERT INTO
stock_movements(product_id,warehouse_id,movement,qty,unit_cost,ref_type,ref_id,ref_no,created_by)
           VALUES ($1,$2,'GRN',$3,$4,'GRN',$5,$6,$7)`,
          [line.product_id, po.warehouse_id, i.qty_received, line.unit_price, grn.id, grn.grn_number, userId]);
        value += i.qty_received * Number(line.unit_price) * (1 + Number(line.tax_rate) / 100);
      }
    }
    await c.query(`UPDATE goods_receipts SET total_value=$1 WHERE id=$2`, [value.toFixed(2), grn.id]);
```

```
    const { rows: [s] } = await c.query(
      `SELECT bool_and(qty_received >= qty_ordered) AS done FROM purchase_order_items WHERE po_id=$1`, [id]);
    await c.query(`UPDATE purchase_orders SET status=$2 WHERE id=$1`, [id, s.done ? 'RECEIVED' : 'PARTIAL']);
    return { ...grn, total_value: +value.toFixed(2) };
  });
}

export const listPOs = (status?: string) => query(
  `SELECT po.*, v.name AS vendor, w.code AS warehouse
     FROM purchase_orders po JOIN vendors v ON v.id=po.vendor_id JOIN warehouses w ON w.id=po.warehouse_id
    WHERE ($1::text IS NULL OR po.status::text=$1) ORDER BY po.id DESC LIMIT 500`, [status ?? null]);

export async function getPO(id: number) {
  const po = await one(
    `SELECT po.*, v.name AS vendor, v.email AS vendor_email, v.gstin AS vendor_gstin, w.name AS warehouse,
            cu.name AS created_by_name, au.name AS approved_by_name
       FROM purchase_orders po JOIN vendors v ON v.id=po.vendor_id JOIN warehouses w ON w.id=po.warehouse_id
       LEFT JOIN users cu ON cu.id=po.created_by LEFT JOIN users au ON au.id=po.approved_by WHERE po.id=$1`,
[id]);
  if (!po) return null;
  const items = await query(
    `SELECT i.*, p.sku, p.name, p.uom FROM purchase_order_items i JOIN products p ON p.id=i.product_id
      WHERE i.po_id=$1 ORDER BY i.id`, [id]);
  const grns = await query(`SELECT * FROM goods_receipts WHERE po_id=$1 ORDER BY id`, [id]);
  return { ...po, items, grns };
}

export async function recordPayment(d: { vendor_id: number; po_id?: number | null; amount: number; paid_on:
string | null; mode: string; reference: string | null }, userId: number) {
  return one(
    `INSERT INTO vendor_payments(vendor_id,po_id,amount,paid_on,mode,reference,created_by)
     VALUES ($1,$2,$3,COALESCE($4::date,CURRENT_DATE),$5,$6,$7) RETURNING *`,
    [d.vendor_id, d.po_id ?? null, d.amount, d.paid_on, d.mode, d.reference, userId]);
}
```

lib/services/sales.ts

```
import { tx, query, one } from '../db';
import { HttpError } from '../auth';

type Line = { product_id: number; qty: number; unit_price: number; tax_rate: number };

export async function createSO(d: { customer_id: number; warehouse_id: number; notes: string | null; items:
Line[] }, userId: number) {
  return tx(async c => {
    const sub = d.items.reduce((s, i) => s + i.qty * i.unit_price, 0);
    const tax = d.items.reduce((s, i) => s + (i.qty * i.unit_price * i.tax_rate) / 100, 0);
    const { rows: [so] } = await c.query(
      `INSERT INTO sales_orders(so_number,customer_id,warehouse_id,notes,subtotal,tax_total,total,created_by)
       VALUES (next_doc_no('SO'),$1,$2,$3,$4,$5,$6,$7) RETURNING *`,
      [d.customer_id, d.warehouse_id, d.notes, sub.toFixed(2), tax.toFixed(2), (sub + tax).toFixed(2), userId]);
    for (const i of d.items)
      await c.query(`INSERT INTO sales_order_items(so_id,product_id,qty,unit_price,tax_rate) VALUES
($1,$2,$3,$4,$5)`,
        [so.id, i.product_id, i.qty, i.unit_price, i.tax_rate]);
    return so;
  });
}

async function lockSO(c: any, id: number) {
  const { rows: [so] } = await c.query(`SELECT * FROM sales_orders WHERE id=$1 FOR UPDATE`, [id]);
  if (!so) throw new HttpError(404, 'Sales order not found');
  const { rows: items } = await c.query(
    `SELECT i.*, p.sku FROM sales_order_items i JOIN products p ON p.id=i.product_id WHERE so_id=$1`, [id]);
  return { so, items };
}

/** Confirm = reserve stock (available = on_hand - reserved) */
export async function confirmSO(id: number) {
  return tx(async c => {
    const { so, items } = await lockSO(c, id);
    if (so.status !== 'DRAFT') throw new HttpError(422, `Cannot confirm a ${so.status} order`);
    for (const i of items) {
      const { rowCount } = await c.query(
```

```
        `UPDATE stock_levels SET qty_reserved = qty_reserved + $1, updated_at=now()
          WHERE product_id=$2 AND warehouse_id=$3 AND qty_on_hand - qty_reserved >= $1`,
        [i.qty, i.product_id, so.warehouse_id]);
      if (!rowCount) throw new HttpError(422, `Insufficient available stock for ${i.sku}`);
    }
    const { rows: [r] } = await c.query(`UPDATE sales_orders SET status='CONFIRMED' WHERE id=$1 RETURNING *`,
[id]);
    return r;
  });
}

/** Dispatch = release reservation + post negative stock movement */
export async function dispatchSO(id: number, userId: number) {
  return tx(async c => {
    const { so, items } = await lockSO(c, id);
    if (so.status !== 'CONFIRMED') throw new HttpError(422, `Only CONFIRMED orders can be dispatched (is
${so.status})`);
    for (const i of items) {
      await c.query(`UPDATE stock_levels SET qty_reserved = qty_reserved - $1 WHERE product_id=$2 AND
warehouse_id=$3`,
        [i.qty, i.product_id, so.warehouse_id]);
      await c.query(
        `INSERT INTO stock_movements(product_id,warehouse_id,movement,qty,ref_type,ref_id,ref_no,created_by)
         VALUES ($1,$2,'DISPATCH',$3,'SO',$4,$5,$6)`,
        [i.product_id, so.warehouse_id, -i.qty, so.id, so.so_number, userId]);
    }
    const { rows: [r] } = await c.query(
      `UPDATE sales_orders SET status='DISPATCHED', dispatched_at=now() WHERE id=$1 RETURNING *`, [id]);
    return r;
  });
}

export async function cancelSO(id: number) {
  return tx(async c => {
    const { so, items } = await lockSO(c, id);
    if (!['DRAFT', 'CONFIRMED'].includes(so.status)) throw new HttpError(422, `Cannot cancel a ${so.status}
order`);
    if (so.status === 'CONFIRMED')
      for (const i of items)
        await c.query(`UPDATE stock_levels SET qty_reserved = qty_reserved - $1 WHERE product_id=$2 AND
warehouse_id=$3`,
          [i.qty, i.product_id, so.warehouse_id]);
    const { rows: [r] } = await c.query(`UPDATE sales_orders SET status='CANCELLED' WHERE id=$1 RETURNING *`,
[id]);
    return r;
  });
}

export const listSOs = () => query(
  `SELECT so.*, c.name AS customer, w.code AS warehouse FROM sales_orders so
     JOIN customers c ON c.id=so.customer_id JOIN warehouses w ON w.id=so.warehouse_id ORDER BY so.id DESC LIMIT
500`);

export async function getSO(id: number) {
  const so = await one(
    `SELECT so.*, c.name AS customer, c.address AS customer_address, w.name AS warehouse
       FROM sales_orders so JOIN customers c ON c.id=so.customer_id JOIN warehouses w ON w.id=so.warehouse_id
WHERE so.id=$1`, [id]);
  if (!so) return null;
  const items = await query(
    `SELECT i.*, p.sku, p.name, p.uom,
            COALESCE(s.qty_on_hand - s.qty_reserved, 0) AS available
       FROM sales_order_items i JOIN products p ON p.id=i.product_id
       LEFT JOIN stock_levels s ON s.product_id=i.product_id AND s.warehouse_id=$2
      WHERE so_id=$1 ORDER BY i.id`, [id, so.warehouse_id]);
  return { ...so, items };
}
```

lib/services/inventory.ts

```
import { tx, query, one } from '../db';

export async function createTransfer(d: { from_warehouse_id: number; to_warehouse_id: number; notes: string |
null;
```

```
  items: { product_id: number; qty: number }[] }, userId: number) {
  return tx(async c => {
    const { rows: [t] } = await c.query(
      `INSERT INTO stock_transfers(transfer_number,from_warehouse_id,to_warehouse_id,notes,created_by)
       VALUES (next_doc_no('TRF'),$1,$2,$3,$4) RETURNING *`, [d.from_warehouse_id, d.to_warehouse_id, d.notes,
userId]);
    for (const i of d.items) {
      await c.query(`INSERT INTO stock_transfer_items(transfer_id,product_id,qty) VALUES ($1,$2,$3)`, [t.id,
i.product_id, i.qty]);
      // OUT at source (trigger validates stock & values at avg cost)
      const { rows: [out] } = await c.query(
        `INSERT INTO stock_movements(product_id,warehouse_id,movement,qty,ref_type,ref_id,ref_no,created_by)
         VALUES ($1,$2,'TRANSFER_OUT',$3,'TRF',$4,$5,$6) RETURNING unit_cost`,
        [i.product_id, d.from_warehouse_id, -i.qty, t.id, t.transfer_number, userId]);
      // IN at destination carrying the same cost
      await c.query(
        `INSERT INTO
stock_movements(product_id,warehouse_id,movement,qty,unit_cost,ref_type,ref_id,ref_no,created_by)
         VALUES ($1,$2,'TRANSFER_IN',$3,$4,'TRF',$5,$6,$7)`,
        [i.product_id, d.to_warehouse_id, i.qty, out.unit_cost, t.id, t.transfer_number, userId]);
    }
    return t;
  });
}

export async function createAdjustment(d: { warehouse_id: number; reason: string; notes: string | null;
  items: { product_id: number; qty_change: number; unit_cost: number }[] }, userId: number) {
  return tx(async c => {
    const { rows: [a] } = await c.query(
      `INSERT INTO stock_adjustments(adj_number,warehouse_id,reason,notes,created_by)
       VALUES (next_doc_no('ADJ'),$1,$2,$3,$4) RETURNING *`, [d.warehouse_id, d.reason, d.notes, userId]);
    for (const i of d.items) {
      let cost = i.unit_cost;
      if (i.qty_change > 0 && !cost) {
        const p = await c.query(`SELECT cost_price FROM products WHERE id=$1`, [i.product_id]);
        cost = Number(p.rows[0]?.cost_price ?? 0);
      }
      await c.query(`INSERT INTO stock_adjustment_items(adjustment_id,product_id,qty_change,unit_cost) VALUES
($1,$2,$3,$4)`,
        [a.id, i.product_id, i.qty_change, cost]);
      await c.query(
        `INSERT INTO
stock_movements(product_id,warehouse_id,movement,qty,unit_cost,ref_type,ref_id,ref_no,note,created_by)
         VALUES ($1,$2,'ADJUSTMENT',$3,$4,'ADJ',$5,$6,$7,$8)`,
        [i.product_id, d.warehouse_id, i.qty_change, i.qty_change > 0 ? cost : 0, a.id, a.adj_number, d.reason,
userId]);
    }
    return a;
  });
}

export const stockLevels = (warehouseId?: number, search?: string) => query(
  `SELECT * FROM v_stock_summary
    WHERE ($1::int IS NULL OR warehouse_id=$1)
      AND ($2::text IS NULL OR sku ILIKE '%'||$2||'%' OR name ILIKE '%'||$2||'%')
    ORDER BY sku, warehouse_code`, [warehouseId ?? null, search || null]);

export const movements = (productId?: number, limit = 200) => query(
  `SELECT m.*, p.sku, p.name, w.code AS warehouse, u.name AS user_name
     FROM stock_movements m JOIN products p ON p.id=m.product_id JOIN warehouses w ON w.id=m.warehouse_id
     LEFT JOIN users u ON u.id=m.created_by
    WHERE ($1::int IS NULL OR m.product_id=$1) ORDER BY m.id DESC LIMIT $2`, [productId ?? null, limit]);

export const listTransfers = () => query(
  `SELECT t.*, f.code AS from_wh, tw.code AS to_wh,
          (SELECT COUNT(*) FROM stock_transfer_items i WHERE i.transfer_id=t.id) AS lines
     FROM stock_transfers t JOIN warehouses f ON f.id=t.from_warehouse_id JOIN warehouses tw ON
tw.id=t.to_warehouse_id
    ORDER BY t.id DESC LIMIT 200`);

export const listAdjustments = () => query(
  `SELECT a.*, w.code AS warehouse, u.name AS user_name,
          (SELECT COUNT(*) FROM stock_adjustment_items i WHERE i.adjustment_id=a.id) AS lines
     FROM stock_adjustments a JOIN warehouses w ON w.id=a.warehouse_id LEFT JOIN users u ON u.id=a.created_by
    ORDER BY a.id DESC LIMIT 200`);
```

```
export async function dashboard() {
  const kpi = await one(`
    SELECT (SELECT COUNT(*) FROM products WHERE active) AS products,
           (SELECT COUNT(*) FROM vendors WHERE active) AS vendors,
           (SELECT COUNT(*) FROM warehouses WHERE active) AS warehouses,
           (SELECT COALESCE(SUM(stock_value),0) FROM v_stock_summary) AS stock_value,
           (SELECT COUNT(*) FROM v_low_stock) AS low_stock,
           (SELECT COUNT(*) FROM purchase_orders WHERE status IN ('APPROVED','PARTIAL')) AS open_pos,
           (SELECT COUNT(*) FROM purchase_orders WHERE status='DRAFT') AS pending_approval,
           (SELECT COUNT(*) FROM sales_orders WHERE status='CONFIRMED') AS to_dispatch,
           (SELECT COALESCE(SUM(balance_due),0) FROM v_vendor_balance) AS payables`);
  const lowStock = await query(`SELECT * FROM v_low_stock ORDER BY qty_available LIMIT 10`);
  const byWarehouse = await query(
    `SELECT warehouse_code, SUM(stock_value) AS value, SUM(qty_on_hand) AS qty FROM v_stock_summary GROUP BY
warehouse_code ORDER BY 1`);
  const recent = await movements(undefined, 10);
  return { kpi, lowStock, byWarehouse, recent };
}
```

lib/client.ts

```
'use client';
import { useCallback, useEffect, useState } from 'react';

export async function apiCall<T = any>(url: string, method = 'GET', body?: unknown): Promise<T> {
  const r = await fetch(url, { method, headers: body ? { 'content-type': 'application/json' } : undefined, body:
body ? JSON.stringify(body) : undefined });
  const data = await r.json().catch(() => ({}));
  if (!r.ok) throw new Error(data.error || `Request failed (${r.status})`);
  return data;
}

export function useData<T = any>(url: string | null) {
  const [data, setData] = useState<T | null>(null);
  const [error, setError] = useState('');
  const reload = useCallback(() => {
    if (!url) return;
    apiCall<T>(url).then(setData).catch(e => setError(e.message));
  }, [url]);
  useEffect(reload, [reload]);
  return { data, error, reload };
}

export const money = (v: unknown) =>
  '₹' + Number(v ?? 0).toLocaleString('en-IN', { minimumFractionDigits: 2, maximumFractionDigits: 2 });
export const qty = (v: unknown) => Number(v ?? 0).toLocaleString('en-IN', { maximumFractionDigits: 3 });
export const date = (v: unknown) => (v ? new Date(String(v)).toLocaleDateString('en-IN') : '—');
```

app/api/adjustments/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { adjustmentSchema } from '@/lib/validators';
import { createAdjustment, listAdjustments } from '@/lib/services/inventory';
export const dynamic = 'force-dynamic';
export const GET = api(async () => { await requireRole(); return listAdjustments(); });
// Adjustments change stock value directly, so only managers/admins may post them
export const POST = api(async req => { const u = await requireRole('ADMIN', 'MANAGER'); return
createAdjustment(adjustmentSchema.parse(await req.json()), u.id); });
```

app/api/auth/login/route.ts

```
import { NextResponse } from 'next/server';
import bcrypt from 'bcryptjs';
import { z } from 'zod';
import { one } from '@/lib/db';
import { COOKIE, signSession } from '@/lib/session';

export async function POST(req: Request) {
  const parsed = z.object({ email: z.string().email(), password: z.string().min(1) }).safeParse(await
req.json());
  if (!parsed.success) return NextResponse.json({ error: 'Email and password required' }, { status: 400 });
```

```
  const u = await one(`SELECT * FROM users WHERE lower(email)=lower($1) AND active`, [parsed.data.email]);
  if (!u || !(await bcrypt.compare(parsed.data.password, u.password_hash)))
    return NextResponse.json({ error: 'Invalid email or password' }, { status: 401 });
  const token = await signSession({ id: u.id, name: u.name, email: u.email, role: u.role });
  const res = NextResponse.json({ id: u.id, name: u.name, role: u.role });
  res.cookies.set(COOKIE, token, { httpOnly: true, sameSite: 'lax', secure: process.env.NODE_ENV ===
'production', path: '/', maxAge: 43200 });
  return res;
}
```

app/api/auth/logout/route.ts

```
import { NextResponse } from 'next/server';
import { COOKIE } from '@/lib/session';
export async function POST() {
  const res = NextResponse.json({ ok: true });
  res.cookies.delete(COOKIE);
  return res;
}
```

app/api/auth/me/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
export const GET = api(async () => requireRole());
export const dynamic = 'force-dynamic';
```

app/api/categories/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { categories } from '@/lib/crud';
import { categorySchema } from '@/lib/validators';
export const dynamic = 'force-dynamic';
export const GET = api(async () => { await requireRole(); return categories.list(); });
export const POST = api(async req => { await requireRole('ADMIN','MANAGER'); return
categories.create(categorySchema.parse(await req.json())); });
```

app/api/customers/[id]/route.ts

```
import { api } from '@/lib/api';
import { requireRole, HttpError } from '@/lib/auth';
import { customers } from '@/lib/crud';
import { customerSchema } from '@/lib/validators';

export const GET = api(async (_req, { params }) => {
  await requireRole();
  const row = await customers.get(Number(params.id));
  if (!row) throw new HttpError(404, 'Not found');
  return row;
});
export const PUT = api(async (req, { params }) => {
  await requireRole('ADMIN','MANAGER','STAFF');
  return customers.update(Number(params.id), customerSchema.partial().parse(await req.json()));
});
```

app/api/customers/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { customers } from '@/lib/crud';
import { customerSchema } from '@/lib/validators';

export const dynamic = 'force-dynamic';
export const GET = api(async req => {
  await requireRole();
  return customers.list(new URL(req.url).searchParams.get('q') ?? undefined);
});
export const POST = api(async req => {
```

```
  await requireRole('ADMIN','MANAGER','STAFF');
  return customers.create(customerSchema.parse(await req.json()));
});
```

app/api/dashboard/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { dashboard } from '@/lib/services/inventory';
export const dynamic = 'force-dynamic';
export const GET = api(async () => { await requireRole(); return dashboard(); });
```

app/api/lookups/route.ts

```
// Dropdown data for forms in one call
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { query } from '@/lib/db';
export const dynamic = 'force-dynamic';
export const GET = api(async () => {
  await requireRole();
  const [products, vendors, warehouses, customers, categories] = await Promise.all([
    query(`SELECT id, sku, name, uom, cost_price, sale_price, tax_rate FROM products WHERE active ORDER BY sku`),
    query(`SELECT id, code, name FROM vendors WHERE active ORDER BY name`),
    query(`SELECT id, code, name FROM warehouses WHERE active ORDER BY code`),
    query(`SELECT id, name FROM customers ORDER BY name`),
    query(`SELECT id, name FROM categories ORDER BY name`),
  ]);
  const vendorPrices = await query(`SELECT product_id, vendor_id, unit_price FROM product_vendors`);
  return { products, vendors, warehouses, customers, categories, vendorPrices };
});
```

app/api/products/[id]/route.ts

```
import { api } from '@/lib/api';
import { requireRole, HttpError } from '@/lib/auth';
import { products } from '@/lib/crud';
import { productSchema } from '@/lib/validators';

export const GET = api(async (_req, { params }) => {
  await requireRole();
  const row = await products.get(Number(params.id));
  if (!row) throw new HttpError(404, 'Not found');
  return row;
});
export const PUT = api(async (req, { params }) => {
  await requireRole('ADMIN','MANAGER');
  return products.update(Number(params.id), productSchema.partial().parse(await req.json()));
});
```

app/api/products/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { products } from '@/lib/crud';
import { productSchema } from '@/lib/validators';

export const dynamic = 'force-dynamic';
export const GET = api(async req => {
  await requireRole();
  return products.list(new URL(req.url).searchParams.get('q') ?? undefined);
});
export const POST = api(async req => {
  await requireRole('ADMIN','MANAGER');
  return products.create(productSchema.parse(await req.json()));
});
```

app/api/purchase-orders/[id]/approve/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { approvePO } from '@/lib/services/purchasing';
export const POST = api(async (_r, { params }) => { const u = await requireRole('ADMIN', 'MANAGER'); return
approvePO(Number(params.id), u.id); });
```

app/api/purchase-orders/[id]/cancel/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { cancelPO } from '@/lib/services/purchasing';
export const POST = api(async (_r, { params }) => { await requireRole('ADMIN', 'MANAGER'); return
cancelPO(Number(params.id)); });
```

app/api/purchase-orders/[id]/receive/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { grnSchema } from '@/lib/validators';
import { receivePO } from '@/lib/services/purchasing';
export const POST = api(async (req, { params }) => {
  const u = await requireRole();
  return receivePO(Number(params.id), grnSchema.parse(await req.json()), u.id);
});
```

app/api/purchase-orders/[id]/route.ts

```
import { api } from '@/lib/api';
import { requireRole, HttpError } from '@/lib/auth';
import { getPO } from '@/lib/services/purchasing';
export const GET = api(async (_r, { params }) => {
  await requireRole();
  const po = await getPO(Number(params.id));
  if (!po) throw new HttpError(404, 'Not found');
  return po;
});
```

app/api/purchase-orders/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { poSchema } from '@/lib/validators';
import { createPO, listPOs } from '@/lib/services/purchasing';
export const dynamic = 'force-dynamic';
export const GET = api(async req => { await requireRole(); return listPOs(new
URL(req.url).searchParams.get('status') ?? undefined); });
export const POST = api(async req => { const u = await requireRole(); return createPO(poSchema.parse(await
req.json()), u.id); });
```

app/api/sales-orders/[id]/cancel/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { cancelSO } from '@/lib/services/sales';
export const POST = api(async (_r, { params }) => { const u = await requireRole(); return
cancelSO(Number(params.id)); });
```

app/api/sales-orders/[id]/confirm/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { confirmSO } from '@/lib/services/sales';
export const POST = api(async (_r, { params }) => { const u = await requireRole(); return
confirmSO(Number(params.id)); });
```

app/api/sales-orders/[id]/dispatch/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { dispatchSO } from '@/lib/services/sales';
export const POST = api(async (_r, { params }) => { const u = await requireRole(); return
dispatchSO(Number(params.id), u.id); });
```

app/api/sales-orders/[id]/route.ts

```
import { api } from '@/lib/api';
import { requireRole, HttpError } from '@/lib/auth';
import { getSO } from '@/lib/services/sales';
export const GET = api(async (_r, { params }) => {
  await requireRole();
  const so = await getSO(Number(params.id));
  if (!so) throw new HttpError(404, 'Not found');
  return so;
});
```

app/api/sales-orders/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { soSchema } from '@/lib/validators';
import { createSO, listSOs } from '@/lib/services/sales';
export const dynamic = 'force-dynamic';
export const GET = api(async () => { await requireRole(); return listSOs(); });
export const POST = api(async req => { const u = await requireRole(); return createSO(soSchema.parse(await
req.json()), u.id); });
```

app/api/stock/movements/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { movements } from '@/lib/services/inventory';
export const dynamic = 'force-dynamic';
export const GET = api(async req => {
  await requireRole();
  const p = new URL(req.url).searchParams.get('product_id');
  return movements(p ? Number(p) : undefined);
});
```

app/api/stock/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { stockLevels } from '@/lib/services/inventory';
export const dynamic = 'force-dynamic';
export const GET = api(async req => {
  await requireRole();
  const sp = new URL(req.url).searchParams;
  return stockLevels(sp.get('warehouse_id') ? Number(sp.get('warehouse_id')) : undefined, sp.get('q') ??
undefined);
});
```

app/api/transfers/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { transferSchema } from '@/lib/validators';
import { createTransfer, listTransfers } from '@/lib/services/inventory';
export const dynamic = 'force-dynamic';
export const GET = api(async () => { await requireRole(); return listTransfers(); });
export const POST = api(async req => { const u = await requireRole(); return
createTransfer(transferSchema.parse(await req.json()), u.id); });
```

app/api/users/[id]/route.ts

```
import bcrypt from 'bcryptjs';
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { one } from '@/lib/db';
import { userSchema } from '@/lib/validators';
export const PUT = api(async (req, { params }) => {
  await requireRole('ADMIN');
  const d = userSchema.partial().parse(await req.json());
  const hash = d.password ? await bcrypt.hash(d.password, 10) : null;
  return one(`UPDATE users SET name=COALESCE($2,name), email=COALESCE($3,email),
role=COALESCE($4::user_role,role),
               active=COALESCE($5,active), password_hash=COALESCE($6,password_hash) WHERE id=$1
              RETURNING id,name,email,role,active`, [Number(params.id), d.name, d.email, d.role, d.active,
hash]);
});
```

app/api/users/route.ts

```
import bcrypt from 'bcryptjs';
import { api } from '@/lib/api';
import { requireRole, HttpError } from '@/lib/auth';
import { query, one } from '@/lib/db';
import { userSchema } from '@/lib/validators';
export const dynamic = 'force-dynamic';
export const GET = api(async () => {
  await requireRole('ADMIN');
  return query(`SELECT id,name,email,role,active,created_at FROM users ORDER BY id`);
});
export const POST = api(async req => {
  await requireRole('ADMIN');
  const d = userSchema.parse(await req.json());
  if (!d.password) throw new HttpError(400, 'Password is required');
  return one(`INSERT INTO users(name,email,password_hash,role,active) VALUES ($1,$2,$3,$4,$5)
              RETURNING id,name,email,role,active`, [d.name, d.email, await bcrypt.hash(d.password, 10), d.role,
d.active]);
});
```

app/api/vendor-payments/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { query } from '@/lib/db';
import { paymentSchema } from '@/lib/validators';
import { recordPayment } from '@/lib/services/purchasing';
export const dynamic = 'force-dynamic';
export const GET = api(async req => {
  await requireRole();
  const v = new URL(req.url).searchParams.get('vendor_id');
  return query(`SELECT vp.*, v.name AS vendor, po.po_number FROM vendor_payments vp JOIN vendors v ON
v.id=vp.vendor_id
                LEFT JOIN purchase_orders po ON po.id=vp.po_id WHERE ($1::int IS NULL OR vp.vendor_id=$1) ORDER
BY vp.id DESC`, [v ? Number(v) : null]);
});
export const POST = api(async req => { const u = await requireRole('ADMIN', 'MANAGER'); return
recordPayment(paymentSchema.parse(await req.json()), u.id); });
```

app/api/vendors/[id]/products/route.ts

```
import { z } from 'zod';
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { query, one } from '@/lib/db';

export const GET = api(async (_r, { params }) => {
  await requireRole();
  return query(`SELECT pv.*, p.sku, p.name FROM product_vendors pv JOIN products p ON p.id=pv.product_id
                WHERE vendor_id=$1 ORDER BY p.sku`, [Number(params.id)]);
});
// Link / update a product supplied by this vendor
```

```
export const POST = api(async (req, { params }) => {
  await requireRole('ADMIN', 'MANAGER');
  const d = z.object({
    product_id: z.coerce.number().int(), vendor_sku: z.string().optional().nullable(),
    unit_price: z.coerce.number().min(0), lead_time_days: z.coerce.number().int().min(0).default(7),
    preferred: z.coerce.boolean().default(false),
  }).parse(await req.json());
  return one(
    `INSERT INTO product_vendors(product_id,vendor_id,vendor_sku,unit_price,lead_time_days,preferred)
     VALUES ($1,$2,$3,$4,$5,$6)
     ON CONFLICT (product_id,vendor_id) DO UPDATE SET vendor_sku=EXCLUDED.vendor_sku,
unit_price=EXCLUDED.unit_price,
       lead_time_days=EXCLUDED.lead_time_days, preferred=EXCLUDED.preferred RETURNING *`,
    [d.product_id, Number(params.id), d.vendor_sku ?? null, d.unit_price, d.lead_time_days, d.preferred]);
});
```

app/api/vendors/[id]/route.ts

```
import { api } from '@/lib/api';
import { requireRole, HttpError } from '@/lib/auth';
import { vendors } from '@/lib/crud';
import { one } from '@/lib/db';
import { vendorSchema } from '@/lib/validators';

export const GET = api(async (_req, { params }) => {
  await requireRole();
  const row = await vendors.get(Number(params.id));
  if (!row) throw new HttpError(404, 'Not found');
  const bal = await one(`SELECT total_received_value, total_paid, balance_due, open_pos FROM v_vendor_balance
WHERE vendor_id=$1`, [row.id]);
  return { ...row, ...bal };
});
export const PUT = api(async (req, { params }) => {
  await requireRole('ADMIN', 'MANAGER');
  return vendors.update(Number(params.id), vendorSchema.partial().parse(await req.json()));
});
```

app/api/vendors/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { vendors } from '@/lib/crud';
import { vendorSchema } from '@/lib/validators';

export const dynamic = 'force-dynamic';
export const GET = api(async req => {
  await requireRole();
  return vendors.list(new URL(req.url).searchParams.get('q') ?? undefined);
});
export const POST = api(async req => {
  await requireRole('ADMIN','MANAGER');
  return vendors.create(vendorSchema.parse(await req.json()));
});
```

app/api/warehouses/[id]/route.ts

```
import { api } from '@/lib/api';
import { requireRole, HttpError } from '@/lib/auth';
import { warehouses } from '@/lib/crud';
import { warehouseSchema } from '@/lib/validators';

export const GET = api(async (_req, { params }) => {
  await requireRole();
  const row = await warehouses.get(Number(params.id));
  if (!row) throw new HttpError(404, 'Not found');
  return row;
});
export const PUT = api(async (req, { params }) => {
  await requireRole('ADMIN');
  return warehouses.update(Number(params.id), warehouseSchema.partial().parse(await req.json()));
});
```

app/api/warehouses/route.ts

```
import { api } from '@/lib/api';
import { requireRole } from '@/lib/auth';
import { warehouses } from '@/lib/crud';
import { warehouseSchema } from '@/lib/validators';

export const dynamic = 'force-dynamic';
export const GET = api(async req => {
  await requireRole();
  return warehouses.list(new URL(req.url).searchParams.get('q') ?? undefined);
});
export const POST = api(async req => {
  await requireRole('ADMIN');
  return warehouses.create(warehouseSchema.parse(await req.json()));
});
```

app/layout.tsx

```
import './globals.css';
export const metadata = { title: 'StockPilot WMS', description: 'Stock, Vendor & Warehouse Management' };
export default function RootLayout({ children }: { children: React.ReactNode }) {
  return <html lang="en"><body>{children}</body></html>;
}
```

app/globals.css

```
:root{--navy:#0B1F4B;--indigo:#3B4FE0;--pink:#E91E7B;--teal:#12b39a;--bg:#f4f6fb;--card:#fff;--line:#e3e7f0;--
muted:#6b7590;--text:#1b2440;--warn:#f59e0b;--danger:#dc2626}
*{box-sizing:border-box}html,body{margin:0;font-family:Inter,system-ui,-apple-system,Segoe UI,Roboto,sans-
serif;color:var(--text);background:var(--bg);font-size:14px}
a{color:var(--indigo);text-decoration:none}
.app{display:grid;grid-template-columns:232px 1fr;min-height:100vh}
.side{background:var(--navy);color:#cdd5f0;padding:18px 12px;position:sticky;top:0;height:100vh;overflow:auto}
.brand{color:#fff;font-weight:800;font-size:18px;padding:4px 10px 18px}.brand span{color:var(--pink)}
.side h6{font-size:10px;letter-spacing:.12em;text-transform:uppercase;color:#7f8bb8;margin:16px 10px 6px}
.side a{display:block;color:#cdd5f0;padding:8px 10px;border-radius:8px}.side
a:hover{background:rgba(255,255,255,.08);color:#fff}
.main{padding:24px 28px;min-width:0}
.topbar{display:flex;justify-content:space-between;align-items:center;margin-bottom:18px}
.topbar h1{font-size:22px;margin:0}
.card{background:var(--card);border:1px solid var(--line);border-radius:12px;padding:16px;margin-bottom:16px}
.grid{display:grid;gap:14px}.g4{grid-template-columns:repeat(auto-fit,minmax(190px,1fr))}.g2{grid-template-
columns:repeat(auto-fit,minmax(320px,1fr))}
.kpi .label{color:var(--muted);font-size:12px}.kpi .val{font-size:24px;font-weight:700;margin-top:4px}
table{width:100%;border-collapse:collapse}th,td{padding:9px 10px;border-bottom:1px solid var(--line);text-
align:left;vertical-align:middle}
th{font-size:11px;text-transform:uppercase;letter-spacing:.06em;color:var(--muted);background:#fafbfe}
td.num,th.num{text-align:right;font-variant-numeric:tabular-nums}
.btn{border:1px solid var(--line);background:#fff;padding:8px 14px;border-radius:8px;cursor:pointer;font-
weight:600;font-size:13px;color:var(--text)}
.btn.primary{background:var(--indigo);border-color:var(--indigo);color:#fff}.btn.danger{color:var(--
danger)}.btn.success{background:var(--teal);border-color:var(--teal);color:#fff}
.btn:disabled{opacity:.5;cursor:not-allowed}.btn.sm{padding:4px 9px;font-size:12px}
input,select,textarea{width:100%;padding:8px 10px;border:1px solid var(--line);border-
radius:8px;font:inherit;background:#fff}
label{display:block;font-size:12px;color:var(--muted);margin-bottom:4px}
.form{display:grid;grid-template-columns:repeat(auto-fit,minmax(200px,1fr));gap:12px}
.badge{display:inline-block;padding:2px 8px;border-radius:999px;font-size:11px;font-
weight:700;background:#eef0f7;color:#475070}
.b-DRAFT{background:#eef0f7}.b-APPROVED,.b-CONFIRMED{background:#e5e8ff;color:var(--indigo)}.b-
PARTIAL{background:#fff4dc;color:#a16207}
.b-RECEIVED,.b-DISPATCHED{background:#dcf7f0;color:#0b7d6b}.b-CANCELLED{background:#fde2e2;color:var(--danger)}
.err{background:#fde2e2;color:#991b1b;padding:10px 12px;border-radius:8px;margin-bottom:12px}
.ok{background:#dcf7f0;color:#0b6b5c;padding:10px 12px;border-radius:8px;margin-bottom:12px}
.modal-bg{position:fixed;inset:0;background:rgba(11,31,75,.45);display:flex;align-items:flex-start;justify-
content:center;padding-top:6vh;z-index:50}
.modal{background:#fff;border-radius:14px;padding:20px;width:min(760px,94vw);max-height:86vh;overflow:auto}
.row{display:flex;gap:8px;align-items:center}.spacer{flex:1}.muted{color:var(--muted)}.right{text-align:right}
.login{min-height:100vh;display:grid;place-items:center;background:linear-gradient(135deg,var(--navy),var(--
indigo))}
.login .card{width:360px;padding:28px}
```

```
.low{color:var(--danger);font-weight:700}
.bar{height:8px;border-radius:4px;background:var(--indigo)}
```

app/login/page.tsx

```
'use client';
import { useState } from 'react';

export default function Login() {
  const [email, setEmail] = useState('admin@wms.local');
  const [password, setPassword] = useState('');
  const [err, setErr] = useState('');
  const [busy, setBusy] = useState(false);
  async function submit(e: React.FormEvent) {
    e.preventDefault(); setBusy(true); setErr('');
    const r = await fetch('/api/auth/login', { method: 'POST', headers: { 'content-type': 'application/json' },
body: JSON.stringify({ email, password }) });
    if (r.ok) location.href = '/'; else { setErr((await r.json()).error); setBusy(false); }
  }
  return (
    <div className="login">
      <form className="card" onSubmit={submit}>
        <h2 style={{ marginTop: 0 }}>Stock<span style={{ color: 'var(--pink)' }}>Pilot</span> WMS</h2>
        <p className="muted">Sign in to manage stock, vendors & warehouses</p>
        {err && <div className="err">{err}</div>}
        <label>Email</label><input value={email} onChange={e => setEmail(e.target.value)} />
        <div style={{ height: 10 }} />
        <label>Password</label><input type="password" value={password} onChange={e =>
setPassword(e.target.value)} />
        <div style={{ height: 16 }} />
        <button className="btn primary" style={{ width: '100%' }} disabled={busy}>{busy ? 'Signing in…' : 'Sign
in'}</button>
      </form>
    </div>
  );
}
```

app/(app)/layout.tsx

```
import Link from 'next/link';
import { requirePageUser } from '@/lib/auth';
import LogoutButton from '@/components/LogoutButton';

export const dynamic = 'force-dynamic';
const nav: [string, [string, string][]][] = [
  ['Overview', [['/', 'Dashboard']]],
  ['Inventory', [['/stock', 'Stock Levels'], ['/stock/movements', 'Stock Ledger'], ['/transfers', 'Transfers'],
['/adjustments', 'Adjustments']]],
  ['Inbound', [['/purchase-orders', 'Purchase Orders'], ['/vendors', 'Vendors'], ['/vendor-payments', 'Vendor
Payments']]],
  ['Outbound', [['/sales-orders', 'Sales / Dispatch'], ['/customers', 'Customers']]],
  ['Masters', [['/products', 'Products'], ['/categories', 'Categories'], ['/warehouses', 'Warehouses'],
['/users', 'Users']]],
];

export default async function AppLayout({ children }: { children: React.ReactNode }) {
  const user = await requirePageUser();
  return (
    <div className="app">
      <aside className="side">
        <div className="brand">Stock<span>Pilot</span></div>
        {nav.map(([h, links]) => (
          <div key={h}><h6>{h}</h6>
            {links.filter(([href]) => href !== '/users' || user.role === 'ADMIN')
              .map(([href, t]) => <Link key={href} href={href}>{t}</Link>)}
          </div>
        ))}
        <div style={{ marginTop: 24, padding: '0 10px', fontSize: 12 }}>
          <div style={{ color: '#fff' }}>{user.name}</div>
          <div style={{ marginBottom: 8 }}>{user.role}</div>
          <LogoutButton />
        </div>
      </aside>
```

```
      <main className="main">{children}</main>
    </div>
  );
}
```

components/MasterPage.tsx

```
'use client';
import { useState } from 'react';
import { apiCall, useData } from '@/lib/client';

export type Field = { name: string; label: string; type?: 'text' | 'number' | 'select' | 'checkbox' | 'textarea'
| 'password' | 'email';
  options?: { value: string | number; label: string }[]; required?: boolean };
export type Column = { key: string; label: string; num?: boolean; render?: (row: any) => React.ReactNode };

/** Generic list + create/edit screen for master data */
export default function MasterPage({ title, endpoint, fields, columns, canEdit = true, defaults = {} }:
  { title: string; endpoint: string; fields: Field[]; columns: Column[]; canEdit?: boolean; defaults?:
Record<string, any> }) {
  const [q, setQ] = useState('');
  const { data, error, reload } = useData<any[]>(`${endpoint}${q ? `?q=${encodeURIComponent(q)}` : ''}`);
  const [editing, setEditing] = useState<Record<string, any> | null>(null);
  const [formErr, setFormErr] = useState('');

  async function save(e: React.FormEvent) {
    e.preventDefault(); setFormErr('');
    const body: Record<string, any> = {};
    for (const f of fields) {
      const v = editing![f.name];
      if (f.type === 'checkbox') body[f.name] = !!v;
      else if (v !== undefined && v !== '') body[f.name] = v;
    }
    try {
      if (editing!.id) await apiCall(`${endpoint}/${editing!.id}`, 'PUT', body);
      else await apiCall(endpoint, 'POST', body);
      setEditing(null); reload();
    } catch (err: any) { setFormErr(err.message); }
  }

  return (
    <>
      <div className="topbar"><h1>{title}</h1>
        <div className="row">
          <input placeholder="Search…" value={q} onChange={e => setQ(e.target.value)} style={{ width: 220 }} />
          {canEdit && <button className="btn primary" onClick={() => setEditing({ ...defaults })}>+ New</button>}
        </div>
      </div>
      {error && <div className="err">{error}</div>}
      <div className="card" style={{ padding: 0, overflow: 'auto' }}>
        <table>
          <thead><tr>{columns.map(c => <th key={c.key} className={c.num ? 'num' : ''}>{c.label}</th>)}{canEdit &&
<th />}</tr></thead>
          <tbody>
            {data?.map(r => (
              <tr key={r.id}>
                {columns.map(c => <td key={c.key} className={c.num ? 'num' : ''}>{c.render ? c.render(r) :
String(r[c.key] ?? '–')}</td>)}
                {canEdit && <td className="right"><button className="btn sm" onClick={() => setEditing({ ...r
})}>Edit</button></td>}
              </tr>
            ))}
            {data && !data.length && <tr><td colSpan={columns.length + 1} className="muted">No records</td></tr>}
          </tbody>
        </table>
      </div>
      {editing && (
        <div className="modal-bg" onClick={() => setEditing(null)}>
          <form className="modal" onClick={e => e.stopPropagation()} onSubmit={save}>
            <h3 style={{ marginTop: 0 }}>{editing.id ? 'Edit' : 'New'} {title.replace(/s$/, '')}</h3>
            {formErr && <div className="err">{formErr}</div>}
            <div className="form">
              {fields.map(f => (
                <div key={f.name} style={f.type === 'textarea' ? { gridColumn: '1/-1' } : undefined}>
```

```
                  <label>{f.label}{f.required && ' *'}</label>
                  {f.type === 'select' ? (
                    <select value={editing[f.name] ?? ''} required={f.required} onChange={e => setEditing({
...editing, [f.name]: e.target.value })}>
                      <option value="">— select —</option>
                      {f.options?.map(o => <option key={o.value} value={o.value}>{o.label}</option>)}
                    </select>
                  ) : f.type === 'checkbox' ? (
                    <input type="checkbox" style={{ width: 'auto' }} checked={!!editing[f.name]} onChange={e =>
setEditing({ ...editing, [f.name]: e.target.checked })} />
                  ) : f.type === 'textarea' ? (
                    <textarea rows={2} value={editing[f.name] ?? ''} onChange={e => setEditing({ ...editing,
[f.name]: e.target.value })} />
                  ) : (
                    <input type={f.type ?? 'text'} step="any" required={f.required} value={editing[f.name] ?? ''}
                      onChange={e => setEditing({ ...editing, [f.name]: e.target.value })} />
                  )}
                </div>
              ))}
            </div>
            <div className="row" style={{ marginTop: 16, justifyContent: 'flex-end' }}>
              <button type="button" className="btn" onClick={() => setEditing(null)}>Cancel</button>
              <button className="btn primary">Save</button>
            </div>
          </form>
        </div>
      )}
    </>
  );
}
```

components/LineItems.tsx

```
'use client';
import { money } from '@/lib/client';

export type Line = { product_id: number | ''; qty: number | ''; unit_price?: number | ''; tax_rate?: number | '';
unit_cost?: number | '' };
type Product = { id: number; sku: string; name: string; uom: string; cost_price: string; sale_price: string;
tax_rate: string };

/** Editable item grid used by PO, SO, Transfer, Adjustment forms */
export default function LineItems({ lines, setLines, products, mode, priceFor }: {
  lines: Line[]; setLines: (l: Line[]) => void; products: Product[];
  mode: 'purchase' | 'sale' | 'qty' | 'adjust'; priceFor?: (productId: number) => number | undefined;
}) {
  const priced = mode === 'purchase' || mode === 'sale';
  const upd = (i: number, patch: Partial<Line>) => setLines(lines.map((l, j) => (j === i ? { ...l, ...patch } :
l)));
  const pick = (i: number, id: number) => {
    const p = products.find(x => x.id === id);
    if (!p) return upd(i, { product_id: '' });
    const price = mode === 'sale' ? Number(p.sale_price) : priceFor?.(id) ?? Number(p.cost_price);
    upd(i, { product_id: id, unit_price: price, tax_rate: Number(p.tax_rate) });
  };
  const total = lines.reduce((s, l) => s + Number(l.qty || 0) * Number(l.unit_price || 0) * (1 +
Number(l.tax_rate || 0) / 100), 0);

  return (
    <div className="card" style={{ padding: 0 }}>
      <table>
        <thead><tr>
          <th style={{ width: '40%' }}>Product</th>
          <th className="num">{mode === 'adjust' ? 'Qty change (+/−)' : 'Qty'}</th>
          {priced && <><th className="num">Unit price</th><th className="num">Tax %</th><th className="num">Line
total</th></>}
          {mode === 'adjust' && <th className="num">Unit cost (for +)</th>}
          <th />
        </tr></thead>
        <tbody>
          {lines.map((l, i) => (
            <tr key={i}>
              <td><select value={l.product_id} onChange={e => pick(i, Number(e.target.value))}>
                <option value="">— product —</option>
```

```
                {products.map(p => <option key={p.id} value={p.id}>{p.sku} · {p.name}</option>)}
              </select></td>
              <td><input type="number" step="any" value={l.qty} onChange={e => upd(i, { qty: e.target.value ===
'' ? '' : Number(e.target.value) })} /></td>
              {priced && <>
                <td><input type="number" step="any" value={l.unit_price} onChange={e => upd(i, { unit_price:
Number(e.target.value) })} /></td>
                <td><input type="number" step="any" value={l.tax_rate} onChange={e => upd(i, { tax_rate:
Number(e.target.value) })} /></td>
                <td className="num">{money(Number(l.qty || 0) * Number(l.unit_price || 0) * (1 +
Number(l.tax_rate || 0) / 100))}</td>
              </>}
              {mode === 'adjust' && <td><input type="number" step="any" value={l.unit_cost ?? ''} onChange={e =>
upd(i, { unit_cost: Number(e.target.value) })} /></td>}
              <td><button type="button" className="btn sm danger" onClick={() => setLines(lines.filter((_, j) =>
j !== i))}>x</button></td>
            </tr>
          ))}
        </tbody>
      </table>
      <div className="row" style={{ padding: 12 }}>
        <button type="button" className="btn sm" onClick={() => setLines([...lines, { product_id: '', qty: ''
}])}>+ Add line</button>
        <div className="spacer" />
        {priced && <b>Grand total: {money(total)}</b>}
      </div>
    </div>
  );
}
```

components/Status.tsx

```
export default function Status({ s }: { s: string }) { return <span className={`badge b-${s}`}>{s}</span>; }
```

components/LogoutButton.tsx

```
'use client';
export default function LogoutButton() {
  return <button className="btn sm" onClick={async () => { await fetch('/api/auth/logout', { method: 'POST' });
location.href = '/login'; }}>Sign out</button>;
}
```

app/(app)/adjustments/page.tsx

```
'use client';
import { useState } from 'react';
import LineItems, { Line } from '@/components/LineItems';
import { apiCall, date, useData } from '@/lib/client';

export default function Adjustments() {
  const { data: lk } = useData<any>('/api/lookups');
  const { data, reload } = useData<any[]>('/api/adjustments');
  const [f, setF] = useState<any>({ warehouse_id: '', reason: 'CYCLE_COUNT', notes: '' });
  const [lines, setLines] = useState<Line[]>([{ product_id: '', qty: '' }]);
  const [err, setErr] = useState(''); const [ok, setOk] = useState('');
  async function submit() {
    setErr(''); setOk('');
    try {
      const a = await apiCall('/api/adjustments', 'POST', { ...f,
        items: lines.filter(l => l.product_id && l.qty).map(l => ({ product_id: l.product_id, qty_change: l.qty,
unit_cost: l.unit_cost || 0 })) });
      setOk(`Adjustment ${a.adj_number} posted`); setLines([{ product_id: '', qty: '' }]); reload();
    } catch (e: any) { setErr(e.message); }
  }
  return (
    <>
      <div className="topbar"><h1>Stock Adjustments</h1></div>
      {err && <div className="err">{err}</div>}{ok && <div className="ok">{ok}</div>}
      <div className="card"><h3 style={{ marginTop: 0 }}>New adjustment <span className="muted" style={{
fontSize: 12 }}>(Manager/Admin only — use + to add stock, − to remove)</span></h3>
        <div className="form">
          <div><label>Warehouse</label><select value={f.warehouse_id} onChange={e => setF({ ...f, warehouse_id:
```

```
e.target.value })}><option value="">—</option>{lk?.warehouses.map((w: any) => <option key={w.id} value={w.id}>
{w.code}</option>)}</select></div>
          <div><label>Reason</label><select value={f.reason} onChange={e => setF({ ...f, reason: e.target.value
})}>{['CYCLE_COUNT', 'DAMAGE', 'EXPIRY', 'FOUND', 'OPENING', 'OTHER'].map(r => <option key={r}>{r}</option>)}
</select></div>
          <div><label>Notes</label><input value={f.notes} onChange={e => setF({ ...f, notes: e.target.value })}
/></div>
        </div><div style={{ height: 12 }} />
        <LineItems lines={lines} setLines={setLines} products={lk?.products ?? []} mode="adjust" />
        <button className="btn primary" onClick={submit}>Post adjustment</button>
      </div>
      <div className="card" style={{ padding: 0 }}><table><thead><tr><th>Adj #</th><th>Date</th><th>WH</th>
<th>Reason</th><th className="num">Lines</th><th>By</th></tr></thead>
        <tbody>{data?.map(a => <tr key={a.id}><td>{a.adj_number}</td><td>{date(a.created_at)}</td><td>
{a.warehouse}</td><td>{a.reason}</td><td className="num">{a.lines}</td><td>{a.user_name}</td></tr>)}</tbody>
</table></div>
    </>
  );
}
```

app/(app)/categories/page.tsx

```
'use client';
import MasterPage from '@/components/MasterPage';
export default function Categories() {
  return <MasterPage title="Categories" endpoint="/api/categories" canEdit
    columns={[{ key: 'id', label: 'ID' }, { key: 'name', label: 'Name' }]}
    fields={[{ name: 'name', label: 'Name', required: true }]} />;
}
```

app/(app)/customers/page.tsx

```
'use client';
import MasterPage from '@/components/MasterPage';
export default function Customers() {
  return <MasterPage title="Customers" endpoint="/api/customers"
    columns={[{ key: 'name', label: 'Name' }, { key: 'email', label: 'Email' }, { key: 'phone', label: 'Phone' },
{ key: 'gstin', label: 'GSTIN' }]}
    fields={[{ name: 'name', label: 'Name', required: true }, { name: 'email', label: 'Email', type: 'email' },
      { name: 'phone', label: 'Phone' }, { name: 'gstin', label: 'GSTIN' }, { name: 'address', label: 'Address',
type: 'textarea' }]} />;
}
```

app/(app)/page.tsx

```
import Link from 'next/link';
import { dashboard } from '@/lib/services/inventory';

const inr = (v: any) => '₹' + Number(v).toLocaleString('en-IN', { maximumFractionDigits: 0 });
const n = (v: any) => Number(v).toLocaleString('en-IN', { maximumFractionDigits: 3 });

export default async function Dashboard() {
  const d = await dashboard();
  const k = d.kpi;
  const maxV = Math.max(1, ...d.byWarehouse.map((w: any) => Number(w.value)));
  const tiles: [string, string, string][] = [
    ['Total stock value', inr(k.stock_value), '/stock'], ['Low-stock items', n(k.low_stock), '/stock'],
    ['POs awaiting approval', n(k.pending_approval), '/purchase-orders'], ['Open POs (to receive)',
n(k.open_pos), '/purchase-orders'],
    ['Orders to dispatch', n(k.to_dispatch), '/sales-orders'], ['Vendor payables', inr(k.payables), '/vendors'],
    ['Active products', n(k.products), '/products'], ['Warehouses', n(k.warehouses), '/warehouses'],
  ];
  return (
    <>
      <div className="topbar"><h1>Dashboard</h1>
        <div className="row"><Link className="btn" href="/purchase-orders/new">+ Purchase order</Link><Link
className="btn primary" href="/sales-orders/new">+ Sales order</Link></div></div>
      <div className="grid g4">{tiles.map(([l, v, h]) => <Link key={l} href={h} className="card kpi" style={{
color: 'inherit' }}><div className="label">{l}</div><div className="val">{v}</div></Link>)}</div>
      <div className="grid g2">
        <div className="card"><h3 style={{ marginTop: 0 }}>Reorder alerts</h3>
          <table><thead><tr><th>SKU</th><th>Product</th><th className="num">Available</th><th
```

```
className="num">Reorder at</th><th>Vendor</th></tr></thead>
            <tbody>{d.lowStock.map((r: any) => <tr key={r.product_id}><td>{r.sku}</td><td>{r.name}</td><td
className="num low">{n(r.qty_available)}</td><td className="num">{n(r.reorder_level)}</td><td>{r.preferred_vendor
?? '—'}</td></tr>)}
              {!d.lowStock.length && <tr><td colSpan={5} className="muted">All items above reorder level</td>
</tr>}</tbody></table>
        </div>
        <div className="card"><h3 style={{ marginTop: 0 }}>Stock value by warehouse</h3>
          {d.byWarehouse.map((w: any) => (
            <div key={w.warehouse_code} style={{ marginBottom: 12 }}>
              <div className="row"><b>{w.warehouse_code}</b><div className="spacer" />{inr(w.value)}</div>
              <div className="bar" style={{ width: `${(Number(w.value) / maxV) * 100}%` }} />
            </div>))}
        </div>
      </div>
      <div className="card"><h3 style={{ marginTop: 0 }}>Recent stock movements</h3>
        <table><thead><tr><th>When</th><th>Type</th><th>Ref</th><th>SKU</th><th>WH</th><th
className="num">Qty</th><th className="num">Balance</th></tr></thead>
          <tbody>{d.recent.map((m: any) => <tr key={m.id}><td>{new Date(m.created_at).toLocaleString('en-IN')}
</td><td><span className="badge">{m.movement}</span></td><td>{m.ref_no ?? '—'}</td><td>{m.sku}</td><td>
{m.warehouse}</td>
            <td className="num" style={{ color: Number(m.qty) < 0 ? 'var(--danger)' : 'var(--teal)' }}>
{Number(m.qty) > 0 ? '+' : ''}{n(m.qty)}</td><td className="num">{n(m.balance_after)}</td></tr>)}</tbody></table>
      </div>
    </>
  );
}
```

app/(app)/products/page.tsx

```
'use client';
import MasterPage from '@/components/MasterPage';
import { money, qty, useData } from '@/lib/client';

export default function Products() {
  const { data: lk } = useData<any>('/api/lookups');
  const cats = (lk?.categories ?? []).map((c: any) => ({ value: c.id, label: c.name }));
  return <MasterPage title="Products" endpoint="/api/products" defaults={{ uom: 'PCS', tax_rate: 18, active: true
}}
    columns={[
      { key: 'sku', label: 'SKU' }, { key: 'name', label: 'Name' },
      { key: 'category_id', label: 'Category', render: r => cats.find((c: any) => c.value ===
r.category_id)?.label ?? '—' },
      { key: 'uom', label: 'UOM' },
      { key: 'cost_price', label: 'Cost', num: true, render: r => money(r.cost_price) },
      { key: 'sale_price', label: 'Price', num: true, render: r => money(r.sale_price) },
      { key: 'reorder_level', label: 'Reorder lvl', num: true, render: r => qty(r.reorder_level) },
      { key: 'active', label: 'Active', render: r => (r.active ? 'Yes' : 'No') },
    ]}
    fields={[
      { name: 'sku', label: 'SKU', required: true }, { name: 'name', label: 'Name', required: true },
      { name: 'category_id', label: 'Category', type: 'select', options: cats },
      { name: 'uom', label: 'Unit of measure' }, { name: 'barcode', label: 'Barcode' }, { name: 'hsn_code',
label: 'HSN code' },
      { name: 'cost_price', label: 'Cost price', type: 'number' }, { name: 'sale_price', label: 'Sale price',
type: 'number' },
      { name: 'tax_rate', label: 'GST %', type: 'number' },
      { name: 'reorder_level', label: 'Reorder level', type: 'number' }, { name: 'reorder_qty', label: 'Reorder
qty', type: 'number' },
      { name: 'active', label: 'Active', type: 'checkbox' },
    ]} />;
}
```

app/(app)/purchase-orders/[id]/page.tsx

```
'use client';
import Link from 'next/link';
import { useEffect, useState } from 'react';
import Status from '@/components/Status';
import { apiCall, date, money, qty, useData } from '@/lib/client';

export default function PODetail({ params }: { params: { id: string } }) {
  const { data: po, reload } = useData<any>(`/api/purchase-orders/${params.id}`);
```

```
  const [rcv, setRcv] = useState<Record<number, { r: string; x: string }>>({});
  const [inv, setInv] = useState('');
  const [err, setErr] = useState(''); const [ok, setOk] = useState('');
  useEffect(() => { setRcv({}); }, [po?.status]);

  const act = async (path: string, body?: any, msg = 'Done') => {
    setErr(''); setOk('');
    try { const r = await apiCall(`/api/purchase-orders/${params.id}/${path}`, 'POST', body); setOk(r.grn_number
? `GRN ${r.grn_number} posted — stock updated` : msg); reload(); }
    catch (e: any) { setErr(e.message); }
  };
  if (!po) return <p>Loading…</p>;
  const canReceive = ['APPROVED', 'PARTIAL'].includes(po.status);
  return (<>
    <div className="topbar"><h1>{po.po_number} <Status s={po.status} /></h1>
      <div className="row">
        <Link className="btn" href="/purchase-orders">← Back</Link>
        <button className="btn" onClick={() => window.print()}>Print</button>
        {po.status === 'DRAFT' && <button className="btn primary" onClick={() => act('approve', undefined, 'PO
approved')}>Approve</button>}
        {['DRAFT', 'APPROVED'].includes(po.status) && <button className="btn danger" onClick={() => act('cancel',
undefined, 'PO cancelled')}>Cancel</button>}
      </div></div>
    {err && <div className="err">{err}</div>}{ok && <div className="ok">{ok}</div>}
    <div className="grid g4">
      <div className="card"><div className="muted">Vendor</div><b>{po.vendor}</b><div className="muted">
{po.vendor_gstin}</div></div>
      <div className="card"><div className="muted">Warehouse</div><b>{po.warehouse}</b></div>
      <div className="card"><div className="muted">Order / Expected</div><b>{date(po.order_date)} →
{date(po.expected_date)}</b></div>
      <div className="card"><div className="muted">Created / Approved by</div><b>{po.created_by_name ?? '—'} /
{po.approved_by_name ?? '—'}</b></div>
    </div>
    <div className="card" style={{ padding: 0 }}>
      <table><thead><tr><th>SKU</th><th>Product</th><th className="num">Ordered</th><th
className="num">Received</th><th className="num">Pending</th><th className="num">Price</th><th
className="num">Tax%</th><th className="num">Amount</th>
        {canReceive && <><th className="num">Receive now</th><th className="num">Rejected</th></>}</tr></thead>
        <tbody>{po.items.map((i: any) => { const pend = Number(i.qty_ordered) - Number(i.qty_received); return (
          <tr key={i.id}><td>{i.sku}</td><td>{i.name}</td><td className="num">{qty(i.qty_ordered)}</td><td
className="num">{qty(i.qty_received)}</td><td className="num">{qty(pend)}</td>
            <td className="num">{money(i.unit_price)}</td><td className="num">{Number(i.tax_rate)}</td><td
className="num">{money(i.qty_ordered * i.unit_price * (1 + i.tax_rate / 100))}</td>
            {canReceive && <><td style={{ width: 110 }}><input type="number" step="any" max={pend} placeholder=
{String(pend)} value={rcv[i.id]?.r ?? ''} onChange={e => setRcv({ ...rcv, [i.id]: { r: e.target.value, x:
rcv[i.id]?.x ?? '' } })} disabled={pend <= 0} /></td>
              <td style={{ width: 90 }}><input type="number" step="any" value={rcv[i.id]?.x ?? ''} onChange={e =>
setRcv({ ...rcv, [i.id]: { r: rcv[i.id]?.r ?? '', x: e.target.value } })} disabled={pend <= 0} /></td></>}
          </tr>); })}</tbody>
        <tfoot><tr><th colSpan={7} className="right">Subtotal / Tax / Total</th><th className="num">
{money(po.subtotal)} / {money(po.tax_total)} / <b>{money(po.total)}</b></th>{canReceive && <th colSpan={2} />}
</tr></tfoot>
      </table>
    </div>
    {canReceive && <div className="card row">
      <input placeholder="Vendor invoice no." value={inv} onChange={e => setInv(e.target.value)} style={{ width:
240 }} />
      <button className="btn" onClick={() => setRcv(Object.fromEntries(po.items.map((i: any) => [i.id, { r:
String(Number(i.qty_ordered) - Number(i.qty_received)), x: '' }])))}>Fill all pending</button>
      <div className="spacer" />
      <button className="btn success" onClick={() => act('receive', { invoice_no: inv, items:
Object.entries(rcv).map(([k, v]) => ({ po_item_id: Number(k), qty_received: Number(v.r || 0), qty_rejected:
Number(v.x || 0) })) })}>Post Goods Receipt (GRN)</button>
    </div>}
    {po.grns.length > 0 && <div className="card"><h3 style={{ marginTop: 0 }}>Goods receipts</h3>
      <table><thead><tr><th>GRN #</th><th>Date</th><th>Invoice</th><th className="num">Value</th></tr></thead>
        <tbody>{po.grns.map((g: any) => <tr key={g.id}><td>{g.grn_number}</td><td>{date(g.received_date)}</td>
<td>{g.invoice_no ?? '—'}</td><td className="num">{money(g.total_value)}</td></tr>)}</tbody></table></div>}
  </>);
}
```

app/(app)/purchase-orders/new/page.tsx

```
'use client';
import { useState } from 'react';
import { useRouter } from 'next/navigation';
import LineItems, { Line } from '@/components/LineItems';
import { apiCall, useData } from '@/lib/client';

export default function NewPO() {
  const router = useRouter();
  const { data: lk } = useData<any>('/api/lookups');
  const [f, setF] = useState<any>({ vendor_id: '', warehouse_id: '', expected_date: '', notes: '' });
  const [lines, setLines] = useState<Line[]>([{ product_id: '', qty: '' }]);
  const [err, setErr] = useState('');
  // Vendor-specific price from product_vendors when available
  const priceFor = (pid: number) => lk?.vendorPrices.find((x: any) => x.product_id === pid && String(x.vendor_id)
=== String(f.vendor_id))?.unit_price;
  async function save() {
    setErr('');
    try {
      const po = await apiCall('/api/purchase-orders', 'POST', { ...f, items: lines.filter(l => l.product_id &&
l.qty) });
      router.push(`/purchase-orders/${po.id}`);
    } catch (e: any) { setErr(e.message); }
  }
  return (<>
    <div className="topbar"><h1>New Purchase Order</h1></div>
    {err && <div className="err">{err}</div>}
    <div className="card form">
      <div><label>Vendor *</label><select value={f.vendor_id} onChange={e => setF({ ...f, vendor_id:
e.target.value })}><option value="">—</option>{lk?.vendors.map((v: any) => <option key={v.id} value={v.id}>
{v.code} · {v.name}</option>)}</select></div>
      <div><label>Deliver to warehouse *</label><select value={f.warehouse_id} onChange={e => setF({ ...f,
warehouse_id: e.target.value })}><option value="">—</option>{lk?.warehouses.map((w: any) => <option key={w.id}
value={w.id}>{w.code} · {w.name}</option>)}</select></div>
      <div><label>Expected date</label><input type="date" value={f.expected_date} onChange={e => setF({ ...f,
expected_date: e.target.value })} /></div>
      <div><label>Notes</label><input value={f.notes} onChange={e => setF({ ...f, notes: e.target.value })} />
</div>
    </div>
    <LineItems lines={lines} setLines={setLines} products={lk?.products ?? []} mode="purchase" priceFor={p => {
const v = priceFor(p); return v ? Number(v) : undefined; }} />
    <button className="btn primary" onClick={save}>Save as draft</button>
  </>);
}
```

app/(app)/purchase-orders/page.tsx

```
'use client';
import Link from 'next/link';
import { useState } from 'react';
import Status from '@/components/Status';
import { date, money, useData } from '@/lib/client';

export default function POList() {
  const [st, setSt] = useState('');
  const { data } = useData<any[]>(`/api/purchase-orders${st ? `?status=${st}` : ''}`);
  return (<>
    <div className="topbar"><h1>Purchase Orders</h1>
      <div className="row"><select value={st} onChange={e => setSt(e.target.value)} style={{ width: 170 }}>
<option value="">All statuses</option>{['DRAFT', 'APPROVED', 'PARTIAL', 'RECEIVED', 'CANCELLED'].map(s => <option
key={s}>{s}</option>)}</select>
        <Link className="btn primary" href="/purchase-orders/new">+ New PO</Link></div></div>
    <div className="card" style={{ padding: 0 }}><table><thead><tr><th>PO #</th><th>Date</th><th>Vendor</th>
<th>Warehouse</th><th>Expected</th><th>Status</th><th className="num">Total</th></tr></thead>
      <tbody>{data?.map(p => <tr key={p.id}><td><Link href={`/purchase-orders/${p.id}`}>{p.po_number}</Link></td>
<td>{date(p.order_date)}</td><td>{p.vendor}</td><td>{p.warehouse}</td><td>{date(p.expected_date)}</td><td><Status
s={p.status} /></td><td className="num">{money(p.total)}</td></tr>)}</tbody></table></div>
  </>);
}
```

app/(app)/sales-orders/[id]/page.tsx

```
'use client';
import Link from 'next/link';
import { useState } from 'react';
import Status from '@/components/Status';
import { apiCall, date, money, qty, useData } from '@/lib/client';

export default function SODetail({ params }: { params: { id: string } }) {
  const { data: so, reload } = useData<any>(`/api/sales-orders/${params.id}`);
  const [err, setErr] = useState(''); const [ok, setOk] = useState('');
  const act = async (a: string, msg: string) => {
    setErr(''); setOk('');
    try { await apiCall(`/api/sales-orders/${params.id}/${a}`, 'POST'); setOk(msg); reload(); } catch (e: any) {
setErr(e.message); }
  };
  if (!so) return <p>Loading…</p>;
  return (<>
    <div className="topbar"><h1>{so.so_number} <Status s={so.status} /></h1>
      <div className="row"><Link className="btn" href="/sales-orders">← Back</Link>
        <button className="btn" onClick={() => window.print()}>Print challan</button>
        {so.status === 'DRAFT' && <button className="btn primary" onClick={() => act('confirm', 'Order confirmed
— stock reserved')}>Confirm & reserve</button>}
        {so.status === 'CONFIRMED' && <button className="btn success" onClick={() => act('dispatch', 'Dispatched
— stock deducted')}>Dispatch</button>}
        {['DRAFT', 'CONFIRMED'].includes(so.status) && <button className="btn danger" onClick={() =>
act('cancel', 'Order cancelled')}>Cancel</button>}
      </div></div>
    {err && <div className="err">{err}</div>}{ok && <div className="ok">{ok}</div>}
    <div className="grid g4">
      <div className="card"><div className="muted">Customer</div><b>{so.customer}</b><div className="muted">
{so.customer_address}</div></div>
      <div className="card"><div className="muted">Warehouse</div><b>{so.warehouse}</b></div>
      <div className="card"><div className="muted">Order date</div><b>{date(so.order_date)}</b></div>
      <div className="card"><div className="muted">Dispatched</div><b>{so.dispatched_at ? new
Date(so.dispatched_at).toLocaleString('en-IN') : '—'}</b></div>
    </div>
    <div className="card" style={{ padding: 0 }}><table><thead><tr><th>SKU</th><th>Product</th><th
className="num">Qty</th><th className="num">Available</th><th className="num">Price</th><th className="num">Tax%
</th><th className="num">Amount</th></tr></thead>
      <tbody>{so.items.map((i: any) => <tr key={i.id}><td>{i.sku}</td><td>{i.name}</td><td className="num">
{qty(i.qty)} {i.uom}</td>
        <td className={`num ${so.status === 'DRAFT' && Number(i.available) < Number(i.qty) ? 'low' : ''}`}>
{qty(i.available)}</td>
        <td className="num">{money(i.unit_price)}</td><td className="num">{Number(i.tax_rate)}</td><td
className="num">{money(i.qty * i.unit_price * (1 + i.tax_rate / 100))}</td></tr>)}</tbody>
      <tfoot><tr><th colSpan={6} className="right">Total</th><th className="num">{money(so.total)}</th></tr>
</tfoot></table></div>
  </>);
}
```

app/(app)/sales-orders/new/page.tsx

```
'use client';
import { useState } from 'react';
import { useRouter } from 'next/navigation';
import LineItems, { Line } from '@/components/LineItems';
import { apiCall, useData } from '@/lib/client';

export default function NewSO() {
  const router = useRouter();
  const { data: lk } = useData<any>('/api/lookups');
  const [f, setF] = useState<any>({ customer_id: '', warehouse_id: '', notes: '' });
  const [lines, setLines] = useState<Line[]>([{ product_id: '', qty: '' }]);
  const [err, setErr] = useState('');
  async function save() {
    setErr('');
    try { const so = await apiCall('/api/sales-orders', 'POST', { ...f, items: lines.filter(l => l.product_id &&
l.qty) }); router.push(`/sales-orders/${so.id}`); }
    catch (e: any) { setErr(e.message); }
  }
  return (<>
    <div className="topbar"><h1>New Sales Order</h1></div>
    {err && <div className="err">{err}</div>}
    <div className="card form">
      <div><label>Customer *</label><select value={f.customer_id} onChange={e => setF({ ...f, customer_id:
```

```
e.target.value })}><option value="">—</option>{lk?.customers.map((c: any) => <option key={c.id} value={c.id}>
{c.name}</option>)}</select></div>
      <div><label>Ship from warehouse *</label><select value={f.warehouse_id} onChange={e => setF({ ...f,
warehouse_id: e.target.value })}><option value="">—</option>{lk?.warehouses.map((w: any) => <option key={w.id}
value={w.id}>{w.code} · {w.name}</option>)}</select></div>
      <div><label>Notes</label><input value={f.notes} onChange={e => setF({ ...f, notes: e.target.value })} />
</div>
    </div>
    <LineItems lines={lines} setLines={setLines} products={lk?.products ?? []} mode="sale" />
    <button className="btn primary" onClick={save}>Save as draft</button>
  </>);
}
```

app/(app)/sales-orders/page.tsx

```
'use client';
import Link from 'next/link';
import Status from '@/components/Status';
import { date, money, useData } from '@/lib/client';
export default function SOList() {
  const { data } = useData<any[]>('/api/sales-orders');
  return (<>
    <div className="topbar"><h1>Sales Orders / Dispatch</h1><Link className="btn primary" href="/sales-
orders/new">+ New order</Link></div>
    <div className="card" style={{ padding: 0 }}><table><thead><tr><th>SO #</th><th>Date</th><th>Customer</th>
<th>Warehouse</th><th>Status</th><th className="num">Total</th></tr></thead>
      <tbody>{data?.map(s => <tr key={s.id}><td><Link href={`/sales-orders/${s.id}`}>{s.so_number}</Link></td>
<td>{date(s.order_date)}</td><td>{s.customer}</td><td>{s.warehouse}</td><td><Status s={s.status} /></td><td
className="num">{money(s.total)}</td></tr>)}</tbody></table></div>
  </>);
}
```

app/(app)/stock/movements/page.tsx

```
'use client';
import { useState } from 'react';
import { money, qty, useData } from '@/lib/client';

export default function Ledger() {
  const [p, setP] = useState('');
  const { data: lk } = useData<any>('/api/lookups');
  const { data } = useData<any[]>(`/api/stock/movements${p ? `?product_id=${p}` : ''}`);
  return (
    <>
      <div className="topbar"><h1>Stock Ledger</h1>
        <select value={p} onChange={e => setP(e.target.value)} style={{ width: 260 }}><option value="">All
products</option>{lk?.products.map((x: any) => <option key={x.id} value={x.id}>{x.sku} · {x.name}</option>)}
</select></div>
      <div className="card" style={{ padding: 0, overflow: 'auto' }}>
        <table><thead><tr><th>#</th><th>Date</th><th>Type</th><th>Reference</th><th>SKU</th><th>WH</th><th
className="num">Qty</th><th className="num">Unit cost</th><th className="num">Balance</th><th>By</th>
<th>Note</th></tr></thead>
          <tbody>{data?.map(m => <tr key={m.id}><td>{m.id}</td><td>{new Date(m.created_at).toLocaleString('en-
IN')}</td><td><span className="badge">{m.movement}</span></td><td>{m.ref_no ?? '—'}</td><td>{m.sku}</td><td>
{m.warehouse}</td>
            <td className="num" style={{ color: Number(m.qty) < 0 ? 'var(--danger)' : 'var(--teal)' }}>
{Number(m.qty) > 0 ? '+' : ''}{qty(m.qty)}</td>
            <td className="num">{money(m.unit_cost)}</td><td className="num">{qty(m.balance_after)}</td><td>
{m.user_name ?? '—'}</td><td>{m.note ?? ''}</td></tr>)}</tbody></table>
      </div>
    </>
  );
}
```

app/(app)/stock/page.tsx

```
'use client';
import { useState } from 'react';
import { money, qty, useData } from '@/lib/client';

export default function Stock() {
  const [wh, setWh] = useState(''); const [q, setQ] = useState('');
```

```
  const { data: lk } = useData<any>('/api/lookups');
  const { data } = useData<any[]>(`/api/stock?${new URLSearchParams({ ...(wh && { warehouse_id: wh }), ...(q && { q }) })}`);
  const total = (data ?? []).reduce((s, r) => s + Number(r.stock_value), 0);
  return (
    <>
      <div className="topbar"><h1>Stock Levels</h1>
        <div className="row">
          <select value={wh} onChange={e => setWh(e.target.value)} style={{ width: 200 }}><option value="">All warehouses</option>{lk?.warehouses.map((w: any) => <option key={w.id} value={w.id}>{w.code} · {w.name}</option>)}</select>
          <input placeholder="Search SKU / name" value={q} onChange={e => setQ(e.target.value)} style={{ width: 200 }} />
        </div></div>
      <div className="card" style={{ padding: 0, overflow: 'auto' }}>
        <table><thead><tr><th>SKU</th><th>Product</th><th>Category</th><th>Warehouse</th><th className="num">On hand</th><th className="num">Reserved</th><th className="num">Available</th><th className="num">Avg cost</th><th className="num">Value</th></tr></thead>
          <tbody>{data?.map(r => <tr key={`${r.product_id}-${r.warehouse_id}`}>
            <td>{r.sku}</td><td>{r.name}</td><td>{r.category ?? '—'}</td><td>{r.warehouse_code}</td>
            <td className="num">{qty(r.qty_on_hand)} {r.uom}</td><td className="num">{qty(r.qty_reserved)}</td>
            <td className={`num ${Number(r.qty_available) <= Number(r.reorder_level) ? 'low' : ''}`}>{qty(r.qty_available)}</td>
            <td className="num">{money(r.avg_cost)}</td><td className="num">{money(r.stock_value)}</td></tr>)}</tbody>
          <tfoot><tr><th colSpan={8} className="right">Total value</th><th className="num">{money(total)}</th></tr></tfoot>
        </table>
      </div>
    </>
  );
}
```

app/(app)/transfers/page.tsx

```
'use client';
import { useState } from 'react';
import LineItems, { Line } from '@/components/LineItems';
import { apiCall, date, useData } from '@/lib/client';

export default function Transfers() {
  const { data: lk } = useData<any>('/api/lookups');
  const { data, reload } = useData<any[]>('/api/transfers');
  const [f, setF] = useState<any>({ from_warehouse_id: '', to_warehouse_id: '', notes: '' });
  const [lines, setLines] = useState<Line[]>([{ product_id: '', qty: '' }]);
  const [err, setErr] = useState(''); const [ok, setOk] = useState('');
  async function submit() {
    setErr(''); setOk('');
    try {
      const t = await apiCall('/api/transfers', 'POST', { ...f, items: lines.filter(l => l.product_id && l.qty) });
      setOk(`Transfer ${t.transfer_number} posted`); setLines([{ product_id: '', qty: '' }]); reload();
    } catch (e: any) { setErr(e.message); }
  }
  return (
    <>
      <div className="topbar"><h1>Stock Transfers</h1></div>
      {err && <div className="err">{err}</div>}{ok && <div className="ok">{ok}</div>}
      <div className="card"><h3 style={{ marginTop: 0 }}>New inter-warehouse transfer</h3>
        <div className="form">
          <div><label>From warehouse</label><select value={f.from_warehouse_id} onChange={e => setF({ ...f, from_warehouse_id: e.target.value })}><option value="">—</option>{lk?.warehouses.map((w: any) => <option key={w.id} value={w.id}>{w.code}</option>)}</select></div>
          <div><label>To warehouse</label><select value={f.to_warehouse_id} onChange={e => setF({ ...f, to_warehouse_id: e.target.value })}><option value="">—</option>{lk?.warehouses.map((w: any) => <option key={w.id} value={w.id}>{w.code}</option>)}</select></div>
          <div><label>Notes</label><input value={f.notes} onChange={e => setF({ ...f, notes: e.target.value })} /></div>
        </div><div style={{ height: 12 }} />
        <LineItems lines={lines} setLines={setLines} products={lk?.products ?? []} mode="qty" />
        <button className="btn primary" onClick={submit}>Post transfer</button>
      </div>
      <div className="card" style={{ padding: 0 }}><table><thead><tr><th>Transfer #</th><th>Date</th><th>From</th><th>To</th><th className="num">Lines</th><th>Notes</th></tr></thead>
```

```
        <tbody>{data?.map(t => <tr key={t.id}><td>{t.transfer_number}</td><td>{date(t.created_at)}</td><td>
{t.from_wh}</td><td>{t.to_wh}</td><td className="num">{t.lines}</td><td>{t.notes ?? ''}</td></tr>)}</tbody>
</table></div>
    </>
  );
}
```

app/(app)/users/page.tsx

```
'use client';
import MasterPage from '@/components/MasterPage';
const roles = ['ADMIN', 'MANAGER', 'STAFF'].map(r => ({ value: r, label: r }));
export default function Users() {
  return <MasterPage title="Users" endpoint="/api/users" defaults={{ role: 'STAFF', active: true }}
    columns={[{ key: 'name', label: 'Name' }, { key: 'email', label: 'Email' }, { key: 'role', label: 'Role' },
      { key: 'active', label: 'Active', render: r => (r.active ? 'Yes' : 'No') }]}
    fields={[{ name: 'name', label: 'Name', required: true }, { name: 'email', label: 'Email', type: 'email',
required: true },
      { name: 'password', label: 'Password (leave blank to keep)', type: 'password' },
      { name: 'role', label: 'Role', type: 'select', options: roles, required: true }, { name: 'active', label:
'Active', type: 'checkbox' }]} />;
}
```

app/(app)/vendor-payments/page.tsx

```
'use client';
import Link from 'next/link';
import { date, money, useData } from '@/lib/client';
export default function Payments() {
  const { data } = useData<any[]>('/api/vendor-payments');
  return (<>
    <div className="topbar"><h1>Vendor Payments</h1><span className="muted">Record payments from a vendor's
page</span></div>
    <div className="card" style={{ padding: 0 }}><table><thead><tr><th>Date</th><th>Vendor</th><th>PO</th>
<th>Mode</th><th>Reference</th><th className="num">Amount</th></tr></thead>
      <tbody>{data?.map(p => <tr key={p.id}><td>{date(p.paid_on)}</td><td><Link href={`/vendors/${p.vendor_id}`}>
{p.vendor}</Link></td><td>{p.po_number ?? '—'}</td><td>{p.mode}</td><td>{p.reference ?? '—'}</td><td
className="num">{money(p.amount)}</td></tr>)}</tbody></table></div>
  </>);
}
```

app/(app)/vendors/[id]/page.tsx

```
'use client';
import Link from 'next/link';
import { useState } from 'react';
import Status from '@/components/Status';
import { apiCall, date, money, useData } from '@/lib/client';

export default function VendorDetail({ params }: { params: { id: string } }) {
  const id = params.id;
  const { data: v, reload: reloadV } = useData<any>(`/api/vendors/${id}`);
  const { data: items, reload: reloadItems } = useData<any[]>(`/api/vendors/${id}/products`);
  const { data: pos } = useData<any[]>(`/api/purchase-orders`);
  const { data: pays, reload: reloadPays } = useData<any[]>(`/api/vendor-payments?vendor_id=${id}`);
  const { data: lk } = useData<any>('/api/lookups');
  const [link, setLink] = useState<any>({ product_id: '', unit_price: '', lead_time_days: 7, preferred: false });
  const [pay, setPay] = useState<any>({ amount: '', mode: 'BANK', reference: '' });
  const [msg, setMsg] = useState(''); const [err, setErr] = useState('');
  const vpos = (pos ?? []).filter(p => String(p.vendor_id) === id);

  const act = async (fn: () => Promise<any>, ok: string) => {
    setErr(''); setMsg('');
    try { await fn(); setMsg(ok); } catch (e: any) { setErr(e.message); }
  };
  if (!v) return <p>Loading…</p>;
  return (
    <>
      <div className="topbar"><h1>{v.name} <span className="muted" style={{ fontSize: 14 }}>{v.code}</span></h1>
<Link className="btn" href="/vendors">← Vendors</Link></div>
      {err && <div className="err">{err}</div>}{msg && <div className="ok">{msg}</div>}
      <div className="grid g4">
```

```
        <div className="card kpi"><div className="label">Purchase orders</div><div className="val">{vpos.length}
</div></div>
        <div className="card kpi"><div className="label">Goods received (incl. tax)</div><div className="val">
{money(v.total_received_value)}</div></div>
        <div className="card kpi"><div className="label">Paid</div><div className="val">{money(v.total_paid)}
</div></div>
        <div className="card kpi"><div className="label">Balance due · terms {v.payment_terms_days}d</div><div
className="val">{money(v.balance_due)}</div></div>
      </div>
      <div className="grid g2">
        <div className="card">
          <h3 style={{ marginTop: 0 }}>Products supplied</h3>
          <table><thead><tr><th>SKU</th><th>Name</th><th className="num">Price</th><th className="num">Lead
days</th><th>Preferred</th></tr></thead>
            <tbody>{items?.map(i => <tr key={i.product_id}><td>{i.sku}</td><td>{i.name}</td><td className="num">
{money(i.unit_price)}</td><td className="num">{i.lead_time_days}</td><td>{i.preferred ? '★' : ''}</td></tr>)}
</tbody></table>
          <div className="form" style={{ marginTop: 12 }}>
            <select value={link.product_id} onChange={e => setLink({ ...link, product_id: e.target.value })}>
              <option value="">— link product —</option>{lk?.products.map((p: any) => <option key={p.id} value=
{p.id}>{p.sku}</option>)}
            </select>
            <input type="number" placeholder="Unit price" value={link.unit_price} onChange={e => setLink({
...link, unit_price: e.target.value })} />
            <input type="number" placeholder="Lead days" value={link.lead_time_days} onChange={e => setLink({
...link, lead_time_days: e.target.value })} />
            <label className="row"><input type="checkbox" style={{ width: 'auto' }} checked={link.preferred}
onChange={e => setLink({ ...link, preferred: e.target.checked })} /> Preferred</label>
            <button className="btn" onClick={() => act(async () => { await apiCall(`/api/vendors/${id}/products`,
'POST', link); reloadItems(); }, 'Product linked')}>Save link</button>
          </div>
        </div>
        <div className="card">
          <h3 style={{ marginTop: 0 }}>Payments</h3>
          <table><thead><tr><th>Date</th><th>Mode</th><th>Ref</th><th className="num">Amount</th></tr></thead>
            <tbody>{pays?.map(p => <tr key={p.id}><td>{date(p.paid_on)}</td><td>{p.mode}</td><td>{p.reference ??
'—'}</td><td className="num">{money(p.amount)}</td></tr>)}</tbody></table>
          <div className="form" style={{ marginTop: 12 }}>
            <input type="number" placeholder="Amount" value={pay.amount} onChange={e => setPay({ ...pay, amount:
e.target.value })} />
            <select value={pay.mode} onChange={e => setPay({ ...pay, mode: e.target.value })}>{['BANK', 'UPI',
'CHEQUE', 'CASH'].map(m => <option key={m}>{m}</option>)}</select>
            <input placeholder="Reference / UTR" value={pay.reference} onChange={e => setPay({ ...pay, reference:
e.target.value })} />
            <button className="btn success" onClick={() => act(async () => { await apiCall('/api/vendor-
payments', 'POST', { ...pay, vendor_id: Number(id) }); setPay({ amount: '', mode: 'BANK', reference: '' });
reloadPays(); reloadV(); }, 'Payment recorded')}>Record payment</button>
          </div>
        </div>
      </div>
      <div className="card">
        <h3 style={{ marginTop: 0 }}>Purchase orders</h3>
        <table><thead><tr><th>PO #</th><th>Date</th><th>Status</th><th className="num">Total</th></tr></thead>
          <tbody>{vpos.map(p => <tr key={p.id}><td><Link href={`/purchase-orders/${p.id}`}>{p.po_number}</Link>
</td><td>{date(p.order_date)}</td><td><Status s={p.status} /></td><td className="num">{money(p.total)}</td>
</tr>)}</tbody></table>
      </div>
    </>
  );
}
```

app/(app)/vendors/page.tsx

```
'use client';
import Link from 'next/link';
import MasterPage from '@/components/MasterPage';

export default function Vendors() {
  return <MasterPage title="Vendors" endpoint="/api/vendors" defaults={{ payment_terms_days: 30, active: true }}
    columns={[
      { key: 'code', label: 'Code', render: r => <Link href={`/vendors/${r.id}`}>{r.code}</Link> },
      { key: 'name', label: 'Name', render: r => <Link href={`/vendors/${r.id}`}>{r.name}</Link> },
      { key: 'contact_person', label: 'Contact' }, { key: 'phone', label: 'Phone' }, { key: 'gstin', label:
'GSTIN' },
```

```
      { key: 'payment_terms_days', label: 'Terms (days)', num: true },
      { key: 'rating', label: 'Rating', num: true, render: r => (r.rating ? `★ ${r.rating}` : '—') },
    ]}
    fields={[
      { name: 'code', label: 'Vendor code', required: true }, { name: 'name', label: 'Name', required: true },
      { name: 'contact_person', label: 'Contact person' }, { name: 'email', label: 'Email', type: 'email' },
      { name: 'phone', label: 'Phone' }, { name: 'gstin', label: 'GSTIN' },
      { name: 'payment_terms_days', label: 'Payment terms (days)', type: 'number' },
      { name: 'rating', label: 'Rating (0-5)', type: 'number' },
      { name: 'address', label: 'Address', type: 'textarea' }, { name: 'active', label: 'Active', type:
'checkbox' },
    ]} />;
}
```

app/(app)/warehouses/page.tsx

```
'use client';
import MasterPage from '@/components/MasterPage';
export default function Warehouses() {
  return <MasterPage title="Warehouses" endpoint="/api/warehouses" defaults={{ active: true }}
    columns={[{ key: 'code', label: 'Code' }, { key: 'name', label: 'Name' }, { key: 'city', label: 'City' },
      { key: 'manager', label: 'Manager' }, { key: 'active', label: 'Active', render: r => (r.active ? 'Yes' :
'No') }]}
    fields={[{ name: 'code', label: 'Code', required: true }, { name: 'name', label: 'Name', required: true },
      { name: 'city', label: 'City' }, { name: 'manager', label: 'Manager' },
      { name: 'address', label: 'Address', type: 'textarea' }, { name: 'active', label: 'Active', type:
'checkbox' }]} />;
}
```